People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Abdelhamid Mehri Constantine 2 University



Faculty of New Information and Communication Technologies

Department of Fundamental Computer Science and its Applications

Graduation Project for the Bachelor's Degree in

Computer Science

Option: **Information Technologies**

Theme

Internship Management & Matching Platform

Supervised by:

Dr. Sid Ahmed Hadri

Prepared by:

Nemour Abderahmene

Cheniti el houceine chiheb eddine

Adoui Akram Abderrahmene

-- June 2026 Session --

Table of Contents

General Introduction 9

Context 9

Problem Statement 9

Contributions 10

Document Organization 10

Chapter 1: Project Context 11

1.1 Introduction 11

1.2 General Context of Internships 11

1.3 Current Situation at the University 12

1.4 Existing Platforms 13

1.4.1 LinkedIn 13

1.4.2 Indeed 14

1.4.3 Emploitic 15

1.4.4 University-Specific Platforms 16

1.5 Conclusion 16

Chapter 2: Preliminary Study and Requirements Specification 18

2.1 Introduction 18

2.2 Project Presentation 18

2.2.1 Project Vision and Goals 18

2.2.2 The Three User Spaces 19

2.2.3 Key Innovations and Expected Impact 20

2.3 Actors Identification 21

2.3.1 Student 21

2.3.2 Company 21

2.3.3 Administrator 21

2.4 Requirements Specification 22

2.4.1 Functional Requirements 22

2.4.2 Non-Functional Requirements 23

2.5 Use Case Diagrams 25

2.5.1 Student Use Case Diagram 25

2.5.2 Company Use Case Diagram 26

2.5.3 Admin Use Case Diagram 27

2.6 Textual Descriptions 28

2.6.1 Student: Apply for an Internship 28

2.6.2 Company: Create an Internship Offer 29

2.6.3 Admin: Manage Internships 30

2.7 Conclusion 32

Chapter 3: Analysis and Design 33

3.1 Introduction 33

3.2 System Class Diagram 33

3.2.1 Stag.io System Class Diagram 34

3.3 Navigation Diagram 35

3.3.1 Student Navigation Diagram 36

3.3.2 Company Navigation Diagram 37

3.3.3 Admin Navigation Diagram 38

3.4 Activity Diagram 38

3.4.1 Activity Diagram: Apply for an Internship 40

3.4.2 Activity Diagram: Create Internship Offer 41

3.4.3 Activity Diagram: Manage Internships 42

3.5 Sequence Diagram 43

3.5.1 Sequence Diagram: Apply for an Internship 44

3.5.2 Sequence Diagram: Create Internship Offer 45

3.5.3 Sequence Diagram: Manage Internships 46

3.6 Transition to Relational Modeling 46

3.6.1 Relational Model 48

3.6.2 Relationships 48

3.7 Conclusion 49

Chapter 4: Implementation 50

4.1 Introduction 50

4.2 Proposed System Architecture 50

4.3 Hardware and Software Resources 50

4.3.1 Hardware Specifications 50

4.3.2 Software Tools and Versions 52

4.4 Implementation Examples 55

4.4.1 JWT Authentication and Role-Based Permissions 55

4.4.2 InternshipOffer Model with Computed Duration 56

4.4.3 Offer Search and Filtering 57

4.4.4 Notification Service 57

4.4.5 PDF Agreement Generation 58

4.5 Database Structure 60

4.6 Application Interfaces 60

4.6.1 Landing Home Page 61

4.6.2 Authentication: Login and Register Page 62

4.6.3 Student Dashboard 63

4.6.4 Student My Application Page 64

4.6.5 Student Edit Profile Page 64

4.6.6 Student Digital CV Page 65

4.6.7 Student Internship Offer Interface 65

4.6.8 Company Dashboard 66

4.6.9 Company Internship Offers Page 67

4.6.10 Company Applicant Management Interface 67

4.6.11 Company Analytics Page 68

4.6.12 Admin Dashboard 68

4.6.13 Admin Manage Applications Page 69

4.6.14 Notifications Page 69

4.7 Conclusion 70

General Conclusion 71

Bibliographic References 73

List of Figures

Figure 1: LinkedIn Interface 13

Figure 2: Indeed Interface 14

Figure 3: Emploitic Interface 15

Figure 4: Comparison of Platforms and Their Features 16

Figure 5: Student Use Case Diagram 25

Figure 6: Company Use Case Diagram 26

Figure 7: Admin Use Case Diagram 27

Figure 8: Student Applies for an Internship (Textual Description) 29

Figure 8: Company Creates an Internship Offer (Textual Description) 30

Figure 9: Admin Manages Internships (Textual Description) 32

Figure 10: Stag.io System Class Diagram 34

Figure 11: Student Navigation Diagram 36

Figure 12: Company Navigation Diagram 37

Figure 13: Admin Navigation Diagram 38

Figure 14: Activity Diagram for 'Apply for an Internship' Case 40

Figure 15: Activity Diagram for 'Create Internship Offer' Case 41

Figure 16: Activity Diagram for 'Manage Internships' Case 42

Figure 17: Sequence Diagram for 'Apply for an Internship' Case 44

Figure 18: Sequence Diagram for 'Create Internship Offer' Case 45

Figure 19: Sequence Diagram for 'Manage Internships' Case 46

Figure 20: Relational Model Diagram 48

Figure 21: Laptop 1 Specifications 51

Figure 22: Laptop 2 Specifications 51

Figure 23: Laptop 3 Specifications 51

Figure 23: Testing Environment 52

Figure 24: Django Logo 52

Figure 25: React Logo 53

Figure 26: PostgreSQL Logo 53

Figure 27: Backend Software Tools 53

Figure 28: Frontend Software Tools 54

Figure 29: Development Tools 54

Figure 30: AstahUML Interface 54

Figure 31: Permission Classes for Role-Based Access Control ...……..... 55

Figure 32: InternshipOffer Model with duration_display Property ……..... 56

Figure 33: Offer Search and Filtering Logic (student/views.py) 57

Figure 34: Centralized Notification Service (core/notifications.py) 58

Figure 35: PDF Agreement Generation Service ……..... 59

Figure 36: Database Tables Overview 60

Figure 36: Landing Home Page 62

Figure 37: Login Page 62

Figure 38: Register Page 63

Figure 39: Student Dashboard 64

Figure 40: Student My Application Page 64

Figure 40: Student Edit Profile Page 65

Figure 41: Student Digital CV Page 65

Figure 42: Student Internship Offer Interface 66

Figure 43: Company Dashboard 66

Figure 44: Company Internship Offers Page 67

Figure 45: Company Applicant Management Interface 67

Figure 46: Company Analytics Page 68

Figure 47: Admin Dashboard 68

Figure 48: Admin Manage Applications Page 69

Figure 49: Notification Page 70

General Introduction:

Internships represent a crucial milestone in every student's academic journey, serving as the first real immersion into the professional world.

1. Context

Internship management is a key bridge between university and industry because it allows students to gain real-world experience and helps companies identify potential talent. It also strengthens university–company collaboration and improves the quality of training. Internships are often a mandatory step in academic programs, and their success directly impacts student employability and the university's relationship with the professional environment. However, in many cases the internship process is still handled in a traditional way: students struggle to find the right offers, companies receive applications without an organized system, and university administration spends a lot of time on validation and paperwork. Communication is often fragmented (emails, messages, paper forms), which makes the process slow, difficult to track, and sometimes unclear for all parties involved. To address these issues, this project proposes a digital platform that centralizes internship search, candidate selection, and administrative procedures.

2. Problem Statement

Students increasingly struggle to find suitable internships that match their skills, location, and goals. At the same time, companies often receive many applications but still struggle to identify the right candidates quickly, especially when profiles are incomplete or not well organized. In parallel, university administrative procedures are often slow and not centralized: validation steps, follow-up, and required documents are handled through manual communication and paperwork. This situation creates multiple issues: delays in processing, lack of transparency about the status of applications, and difficulty monitoring placements across departments or academic years. Without a centralized system, it becomes hard to answer basic questions such as: Who applied? Who was accepted? Which documents are missing? What is the current status of each internship request? Therefore, the main need is a centralized platform that improves matching between students and offers, organizes application tracking, and reduces administrative workload through automation and document generation.

3. Contributions

This project aims to deliver several key contributions to improve the management of university internships.

- The development of a full-stack web application, named Stag.io, designed to centralize and streamline the internship management process.
- The implementation of a matching mechanism between internship opportunities and student profiles based on skills-oriented filtering criteria.
- The design of three dedicated user interfaces—Student, Company, and Administration—supporting functionalities such as application submission, candidate evaluation and decision-making (acceptance or rejection), and administrative validation.
- The digitalization of administrative procedures through the automated generation of official documents, such as internship agreements and conventions.
- The integration of basic statistical and reporting features, enabling the monitoring of key indicators, such as the number of students successfully placed in internships versus those still seeking placement.

4. Document Organization

This report is organized as follows: Chapter 1 presents the project context and a brief review of existing solutions. Chapter 2 presents the preliminary study, the system actors, the requirements (functional and non-functional), and the use cases with detailed textual descriptions. Chapter 3 covers the analysis and design phase including class diagrams, sequence diagrams, and activity diagrams. Chapter 4 describes the implementation details, technologies used, and presents the user interfaces of our application. Finally, we conclude with a summary of achievements and future perspectives.

Chapter 1: Project Context

1.1 Introduction

Before designing any solution, it is important to understand the problem fully. This chapter explores the background of our project by looking at the role of internships, how they are currently managed at the university, and what similar platforms already exist. This analysis will help us identify the gaps that Stag.io aims to fill.

1.2 General Context of Internships

An internship is practical work in a student's field, bridging academic knowledge with real-world experience. It typically lasts one to six months.

The concept of internships has evolved significantly over the years. What once was simply a short observation period has now become an integral part of higher education curricula worldwide. In many countries, including Algeria, internships are mandatory for graduation in most fields, particularly in technology and engineering disciplines. This shift reflects the growing recognition that theoretical knowledge alone is insufficient to prepare students for the demands of the modern workforce.

For students, internships provide numerous benefits. First, they offer hands-on experience in their chosen field, allowing students to apply what they have learned in classrooms to real-world situations. Second, they help students develop both technical skills specific to their domain and soft skills such as communication, teamwork, and problem-solving. Third, internships enable students to build professional networks that can lead to future job opportunities. Fourth, they give students a clearer understanding of their career interests and goals. Finally, students with internship experience are significantly more likely to find employment after graduation compared to those without such experience.

For companies, internships are equally valuable. They provide access to fresh talent and new perspectives, as students often bring knowledge of the latest technologies and methodologies. Companies can evaluate potential future employees in a real work environment before committing to a permanent hire. Interns can contribute to short-term projects and help with workload during peak periods. Additionally, companies that offer internships strengthen their relationship with universities and contribute to the development of the future workforce. Many

organizations use internship programs as their primary recruitment pipeline for entry-level positions.

For universities, internships serve multiple purposes. They strengthen relationships with industry partners, which can lead to research collaborations, guest lectures, and equipment donations. Internships improve the quality of education by keeping faculty aware of current industry practices and needs. They increase graduate employability rates, which enhances the university's reputation and attracts prospective students. Internships also help universities fulfill their mission of preparing students for successful careers and contributing to economic development.

Despite these benefits, the internship process remains challenging for students, companies, and universities. A centralized platform is needed to streamline the entire process.

1.3 Current Situation at the University

In many universities, internship management remains manual and paper-based. Students must find opportunities on their own, with no centralized platform to discover available positions.

When a student finds an internship, they must obtain a paper agreement form requiring multiple signatures from the student, company supervisor, academic supervisor, department head, and sometimes the dean. Each signature requires physical presence, forcing repeated trips between the university and company.

This manual process is time-consuming, often taking days or weeks. It lacks transparency, is prone to errors like lost forms, and creates unnecessary bureaucracy for everyone involved.

Communication between students, companies, and the university is fragmented. Students use email, phone calls, or messaging apps to communicate with companies. Companies contact the university through various channels. The administration communicates with students through email or bulletin boards. This fragmentation leads to missed messages, delayed responses, and general confusion about the status of internship applications and approvals.

The COVID-19 pandemic highlighted the weaknesses of this manual system. When universities closed and remote work became necessary, the paper-based approval process became nearly impossible. Students could not physically collect signatures, and the administration struggled to process internship documents remotely. This situation demonstrated the urgent need for a digital solution that can handle the entire internship lifecycle online.

In summary, the current internship management system at our university is outdated, inefficient, and poorly suited to the needs of students, companies, and administrators. The manual processes create delays, lack of transparency, and administrative burden for all parties. These challenges provide a strong motivation for developing a digital platform that can automate and streamline internship management.

1.4 Existing Platforms

Before designing our own solution, it is important to examine existing platforms that attempt to solve similar problems. Several websites and applications already connect students with internship opportunities, though none fully address all the needs identified in our context.

1.4.1 LinkedIn

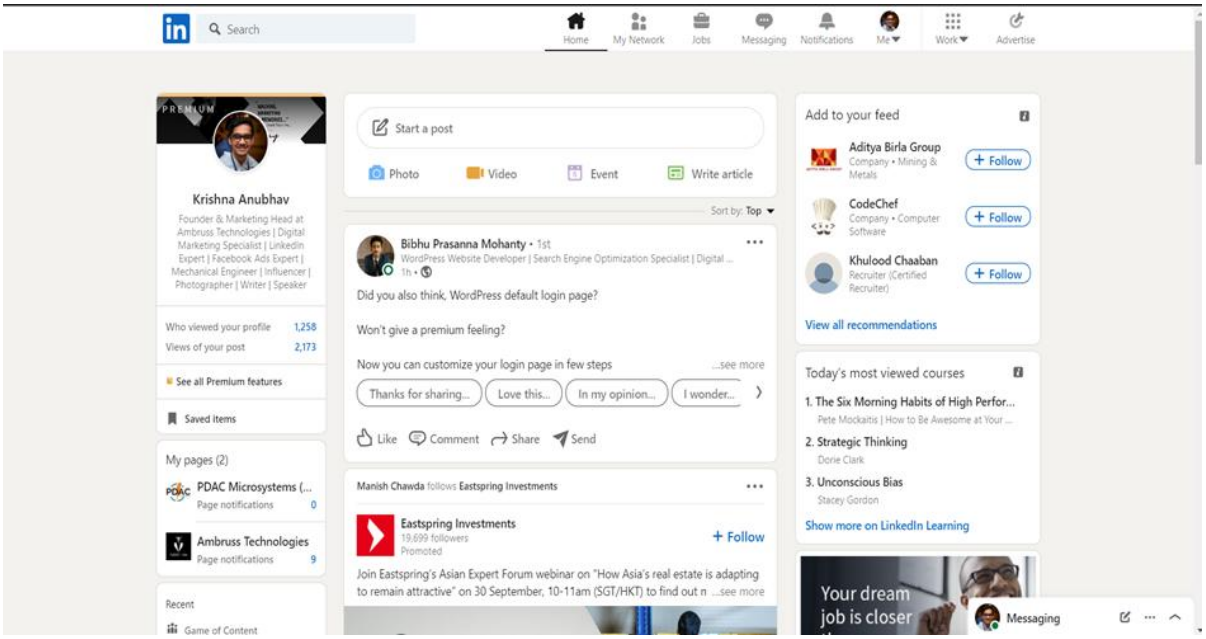


Figure 1: LinkedIn Interface

LinkedIn is the world's largest professional networking platform. It allows users to create professional profiles, connect with colleagues and industry professionals, and search for job and internship opportunities. Companies can post openings and receive applications directly through the platform.

Strengths of LinkedIn include its large user base, professional networking features, and integration with company pages. Students can showcase their skills, experience, and education

in a standardized format. Recruiters can search for candidates based on keywords, location, and other filters.

However, LinkedIn has several limitations for our specific context. It is not designed specifically for internships and does not integrate with university administrative processes. There is no document generation feature for internship agreements. Universities cannot track which students have secured internships or monitor their progress. The platform is also general-purpose, so internship listings compete with thousands of full-time job postings.

1.4.2 Indeed

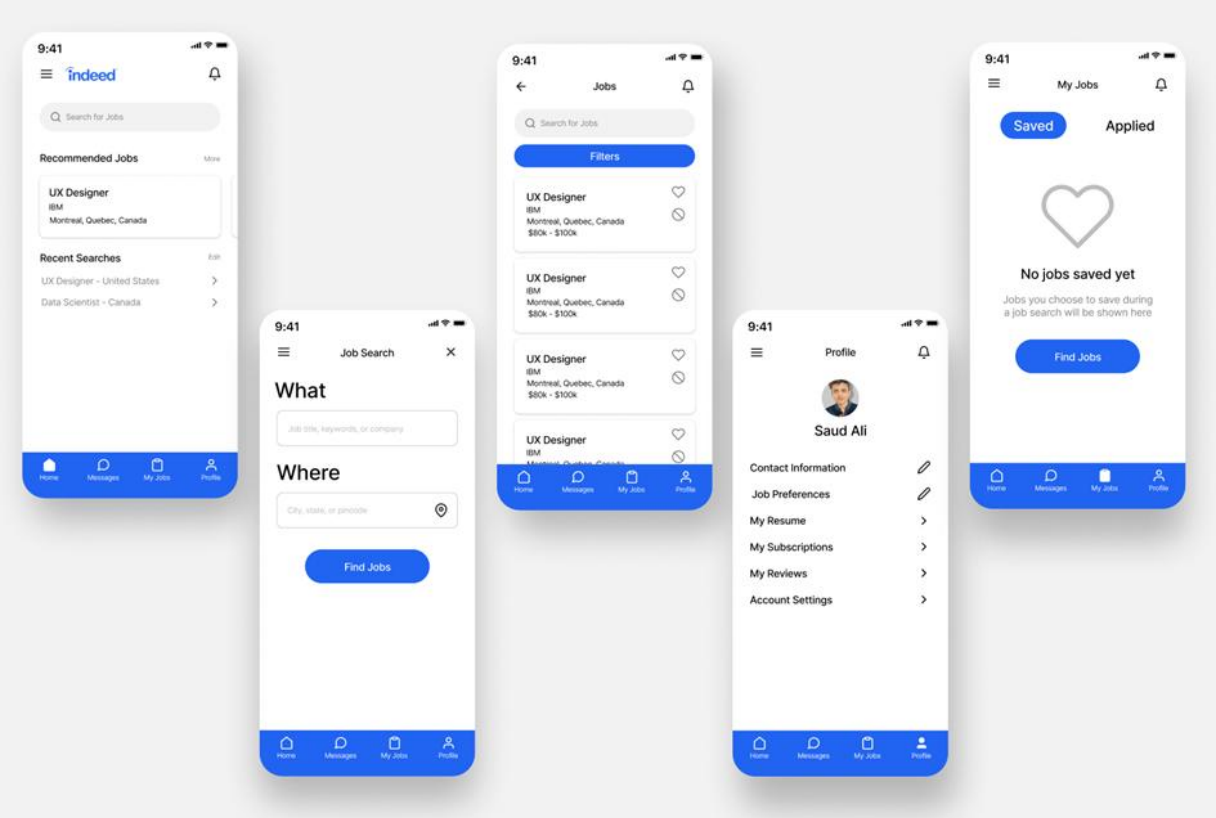


Figure 2: Indeed Interface

Indeed is a job search engine that aggregates listings from thousands of sources, including company websites, job boards, and recruiters. Students can search for internships using keywords and location filters, then apply directly through the platform or be redirected to the employer's website.

Indeed's strengths include its massive database of opportunities and simple search interface. It is free for job seekers and widely used by employers. Students can upload their CVs and apply to multiple positions quickly.

The limitations of Indeed are similar to those of LinkedIn. It is not tailored to the internship process specifically. There is no integration with university administration. Universities cannot verify which students have applied to or accepted internships. Document generation and approval workflows are completely absent.

1.4.3 Emploitic

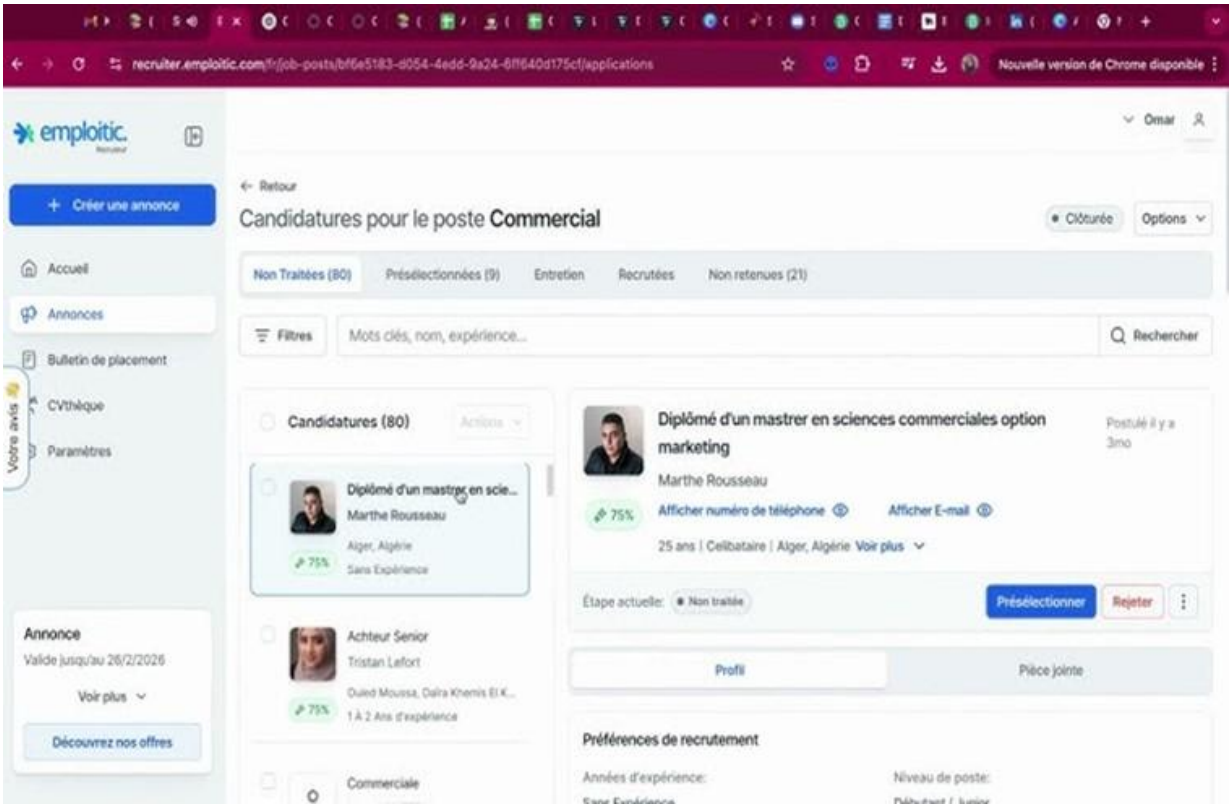


Figure 3: Emploitic Interface

Emploitic is an Algerian job platform that focuses on the local market. It includes job and internship listings for Algerian students and professionals. Companies can post opportunities, and candidates can upload their CVs and apply.

As a local platform, Emploitic better understands the Algerian context and market. Listings are relevant to local students, and the platform is available in both French and Arabic. It has partnerships with some Algerian universities and companies.

However, Emploitic still does not address the administrative side of internships. There is no feature for generating internship agreements or tracking the approval process. Universities cannot use it to monitor student placements or generate statistics. The platform focuses on the matching phase only, leaving the administrative workflow untouched.

1.4.4 University-Specific Platforms

Some universities have developed their own internal platforms for internship management. These systems typically allow students to submit their internship information, upload documents, and track approval status. Department heads can view lists of students and validate their placements.

The main advantage of university-specific platforms is that they address the administrative workflow directly. They can generate the required documents and track students through the approval process. They provide administrators with visibility into internship placements.

The limitation of these platforms is that they often lack the matching component. Students must still find internships on their own through external channels. The platforms are typically built as administrative tools rather than comprehensive solutions that connect students with companies. They also tend to be technically outdated, with poor user interfaces and limited features.

Platform	Matching	Document Generation	Admin Dashboard	University Integration	Local Conte
LinkedIn	NO	NO	NO	NO	NO
Indeed	NO	NO	NO	NO	NO
Emploitic	YES	NO	NO	Partial	YES
Stag.io	YES	YES	YES	YES	YES

Figure 4: Comparison of Platforms and Their Features

1.5 Conclusion

This chapter has established the foundation for our project by exploring the context in which internships operate and the challenges that currently exist.

We began by examining the general importance of internships for students, companies, and universities. Internships serve as a critical bridge between academic learning and professional practice, providing students with hands-on experience, companies with fresh talent, and universities with stronger industry connections.

We then analyzed the current situation at our university, identifying significant problems with the manual, paper-based system. Students struggle to find opportunities and navigate complex

approval processes. Companies face difficulties managing applications and waiting for documents. The administration lacks visibility into internship placements and spends excessive time on paperwork.

Finally, we reviewed existing platforms such as LinkedIn, Indeed, and Emploitic. While these platforms help with the matching phase, none address the full lifecycle of internships including document generation, administrative validation, and university tracking. University-specific platforms handle administration but lack matching features.

This analysis reveals a clear gap: there is no integrated solution that combines internship matching with administrative workflow automation. Students, companies, and universities all suffer from this fragmentation.

The next chapter will build on this understanding by defining the specific requirements for our proposed solution, Stag.io. We will identify the actors, list functional and non-functional requirements, and describe the use cases that will guide our design and implementation.

Chapter 2: Preliminary Study and Requirements Specification

2.1 Introduction

After establishing the context and identifying the gaps in existing solutions in Chapter 1, this chapter focuses on defining exactly what our system Stag.io must do. We will present the project's vision, identify the different actors who will interact with the platform, and list the functional and non-functional requirements. Finally, we will describe the main use cases with detailed textual descriptions and diagrams. This chapter serves as the blueprint that will guide the design and implementation phases of our project.

2.2 Project Presentation

Stag.io is a full-stack web application designed to solve the internship management problem by creating a centralized platform that connects students, companies, and university administration. The name "Stag.io" combines "Stage" (the French word for internship) with ".io" (a common domain extension for tech platforms), reflecting both its purpose and its modern, digital nature. The platform aims to replace the current manual, fragmented, and time-consuming processes with an automated, transparent, and efficient digital solution.

2.2.1 Project Vision and Goals

The vision behind Stag.io is simple but ambitious: to create a seamless experience where students can find internships that match their skills, companies can discover qualified candidates efficiently, and administrators can manage the entire process with minimal effort and maximum visibility. Rather than forcing users to navigate multiple systems, endless email chains, and manual paperwork, Stag.io brings everything into one unified platform.

The primary goals of the project are:

First, to facilitate matching between students and internship opportunities. Students often struggle to find positions that align with their skills and interests. Companies receive many applications but lack efficient tools to identify the best candidates. Stag.io addresses this by implementing a skill-based filtering system that connects the right students with the right opportunities.

Second, to digitize administrative procedures. The current paper-based system for internship agreements is slow, prone to errors, and difficult to track. Stag.io automates document generation, creating official internship agreements and certificates automatically from data already in the system.

Third, to provide centralized tracking and visibility. University administrators currently have limited visibility into which students have found internships, where they are placed, and what their status is. Stag.io provides dashboards and reporting tools that give administrators a complete picture of the internship ecosystem.

Fourth, to reduce workload for all parties. Students should focus on their internships not on chasing signatures. Companies should focus on evaluating candidates, not on processing paperwork. Administrators should focus on supporting students, not on managing documents. Stag.io automates the repetitive tasks so everyone can focus on what matters.

2.2.2 The Three User Spaces

The platform is built around three distinct spaces, each designed for the specific needs of its users:

The Student Space provides a comprehensive environment for students to manage their internship journey. Students begin by creating detailed profiles that showcase their education, technical skills, and professional experience. They can upload their CV, link to their GitHub or portfolio, and select specific technical skills from a predefined list including React, Java, Python, and other relevant technologies. Once their profile is complete, students can search for internship opportunities using powerful filters based on location, required technologies, internship type, duration, and company. When they find interesting positions, they can apply with a single click, sending their profile to the company. Throughout the process, students can track the status of their applications, seeing which companies have viewed their profile, which have accepted them, and which have moved forward with other candidates. When an internship is confirmed, all necessary documents are generated automatically and made available for download.

The Company Space provides tools for organizations to manage their internship recruitment. Companies begin by creating detailed profiles that include their logo, description, location, and contact information. This company profile serves as their identity on the platform and helps students learn about potential employers. From their dashboard, companies can create, modify,

and delete internship offers. Each offer can include detailed descriptions, required skills, preferred duration, start dates, and any other relevant information. When students apply, companies see their applications in an organized dashboard with access to each candidate's full profile, CV, and skills. Companies can accept candidates who seem promising, which automatically triggers the administrative procedure, or refuse candidates who are not a good fit. Throughout the process, companies can communicate with candidates and track the status of their internship postings.

The Administration Space provides university staff with complete visibility and control over the internship process. Department heads and internship office staff can see comprehensive dashboards showing all students, their current status, and their placement information. When a company accepts a student, administrators receive notifications and can review the proposed internship before validation. With a single click, they can validate the internship, which triggers automatic generation of the official Internship Agreement. The system fills all required information from the student, company, and university data already in the system, creating a professional digital document ready for signatures. Administrators can also access statistics and reports showing how many students are placed versus unplaced, which companies are hosting students, and other valuable insights for departmental planning and reporting.

2.2.3 Key Innovations and Expected Impact

Stag.io stands out from existing solutions through its integrated approach to internship management. While platforms like LinkedIn and Indeed focus mainly on matching, and university systems handle administrative tasks separately, Stag.io combines the entire internship lifecycle into a single platform. From searching opportunities to generating signed agreements, all processes are centralized, creating a smoother and more transparent experience for students, companies, and administrators.

The platform introduces smart features that go beyond basic automation. A skill-based matching system connects students with relevant opportunities and companies with suitable candidates. A notification system keeps users informed in real time, while automated document generation ensures consistency and reduces errors by using reliable data sources instead of manual input.

Built as a modern, scalable web application, Stag.io is accessible from any device and designed with security and maintainability in mind. As adoption grows, it can handle increasing demand while protecting sensitive data and supporting future improvements.

The expected impact is significant. Students will spend less time searching and more time focusing on their learning. Companies will recruit more efficiently, and administrators will reduce manual workload while gaining better oversight. Additionally, Stag.io provides a strong foundation for future enhancements such as mobile access, advanced matching algorithms, and system integrations, making it a long-term solution for improving internship management.

2.3 Actors Identification

In any software system, actors are the entities that interact with the system. They can be human users or external systems. For Stag.io, we have identified three main actors. Each actor has specific roles, permissions, and goals within the system.

2.3.1 Student

The Student is the primary end-user of the platform. Any enrolled university student seeking an internship can register and use the system. The student's main goal is to find a suitable internship opportunity that matches their skills, location preferences, and career goals. The student can register and create a personal profile, upload their CV and link to GitHub or portfolio, and select technical skills from a predefined list such as React, Java, or Python. They can search for internship offers using filters, view detailed information about offers, and apply to opportunities. Throughout the process, students can track the status of their applications, receive notifications about updates, and download generated documents once an internship is validated.

2.3.2 Company

The Company actor represents any organization, business, or institution that offers internship opportunities to students. This includes private companies, public institutions, startups, and any other entity looking to recruit interns. Companies can register and create a company profile with logo, description, and location. They can post new internship offers with detailed descriptions, and modify or delete existing offers when needed. Companies have access to a list of students who have applied to their offers, allowing them to view student profiles, CVs, and skills. They can accept or reject student applications, receive notifications when new applications are submitted, and track the status of their internship postings throughout the recruitment process.

2.3.3 Administrator

The Administrator actor represents university staff responsible for overseeing and validating the internship process. This includes department heads, internship office staff, and academic supervisors who need to monitor student placements and manage administrative documents. Administrators can view all students and their internship status, as well as all companies and their posted offers. They receive notifications when a company accepts a student and can review internship details before validation. With a single click, they can validate internship agreements and automatically generate official documents such as the Internship Agreement. Administrators also have access to statistics and reports on placements, allowing them to monitor placed versus unplaced students across the university.

2.4 Requirements Specification

This section defines the complete set of requirements that our system, Stag.io, must satisfy. The requirements are divided into two categories: functional requirements, which describe what the system must do, and non-functional requirements, which describe how the system should behave in terms of performance, security, and usability.

2.4.1 Functional Requirements

For the Student Actor:

- Register using university email.
- Create and edit personal profile.
- Upload CV in digital format.
- Add GitHub or portfolio link.
- Select technical skills from predefined list.
- Search internship offers by location, domain, type.
- View detailed offer information.
- Apply to offers with one click.
- Track application status.
- Receive notifications on status changes.
- Download generated internship agreement.

For the Company Actor:

- Register and create company profile with logo, description, location.
- Create, modify, and delete internship offers.
- View dashboard of applicants.
- Access student profiles, CVs, and skills.
- Accept or reject applications.
- Receive notifications for new applications.
- Automatically trigger admin procedure when accepting a student.

For the Administrator Actor:

- View dashboard of all students and their status.
- View dashboard of all companies and their offers.
- Receive notifications when a student is accepted.
- Review and validate internship agreements.
- Generate official Internship Agreement PDF automatically.
- View statistics of placed vs unplaced students.
- Generate reports by department or academic year.

2.4.2 Non-Functional Requirements

Performance Requirements:

- Rapidly respond to user requests.
- Support multiple users simultaneously.
- Generate digital documents in a short period.
- Display search results almost instantaneously.

Security Requirements:

- Encrypt passwords with secure hashing.

- Protect sessions with secure tokens.
- Restrict access based on user role.
- Validate all inputs to prevent attacks.
- Use HTTPS for all communications.
- Store sensitive data securely.

Usability Requirements:

- Provide clean and intuitive interface.
- Be responsive on all devices.
- Display clear error messages.
- Show confirmation messages for important actions.
- Use consistent design throughout.

Reliability Requirements:

- Back up data daily.
- Validate all generated documents.
- Handle concurrent updates without corruption.

Maintainability Requirements:

- Use modular architecture for easy updates.
- Document code and follow standards.
- Use version control for development.

Scalability Requirements:

- Allow scaling as user base grows.
- Design database for efficient data handling.

2.5 Use Case Diagrams

A use case diagram (UCD) is a simplified visual representation of the interactions between users and a software system. It shows the different actions that users can perform and how they interact with the system to achieve their goals.

The key elements of a use case diagram include:

➢ Actor: represents a user or external system that interacts with the system you are modeling.

➢ Use case: represents a specific feature or action that the system offers to its users.

➢ Relationship: used to show the interaction between an actor and a specific use case

2.5.1 Student Use Case diagram:

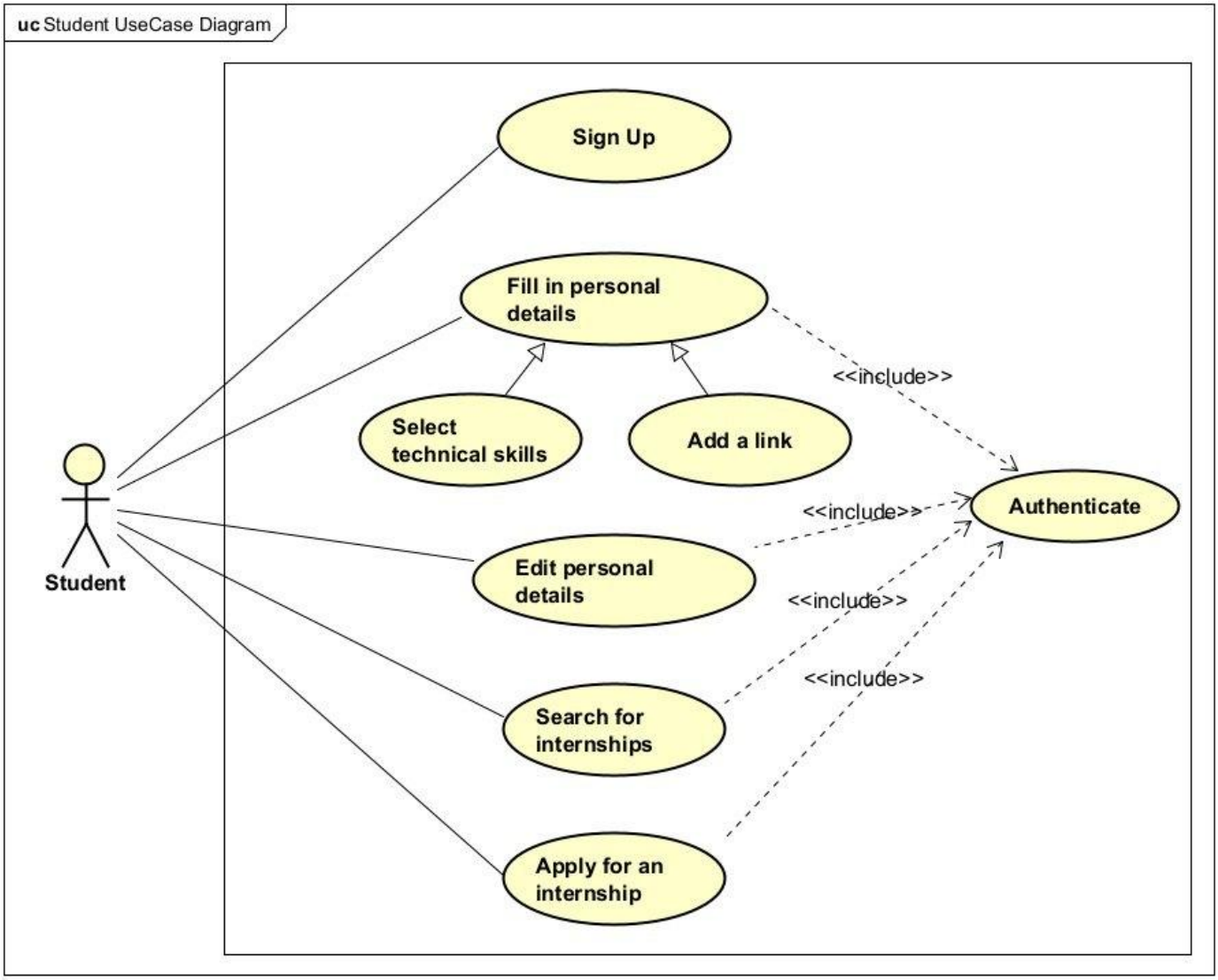


Figure 5: Student Use Case diagram

2.5.2 Company Use Case diagram:

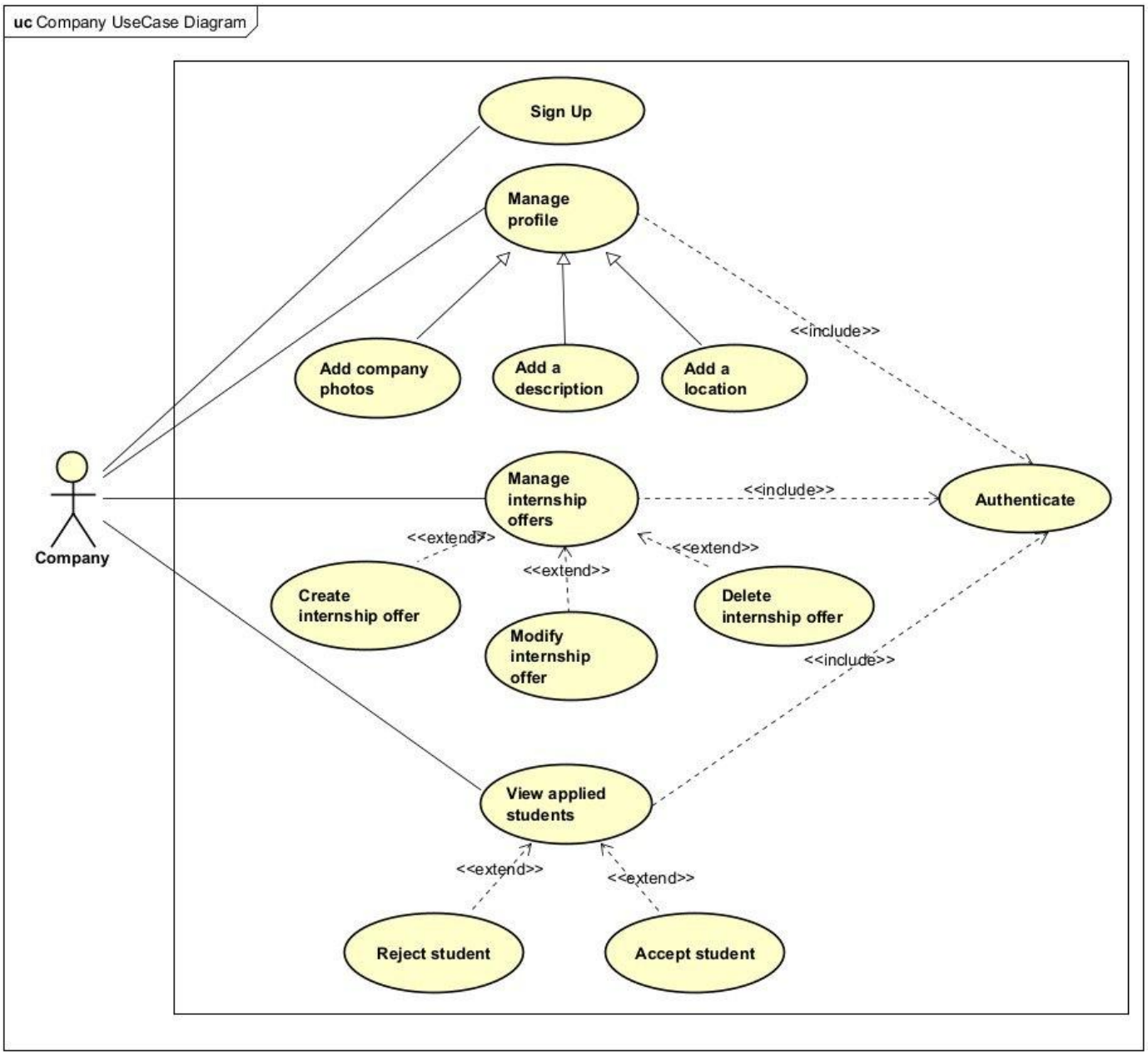


Figure 6: Company Use Case diagram

2.5.3 Admin Use Case diagram:

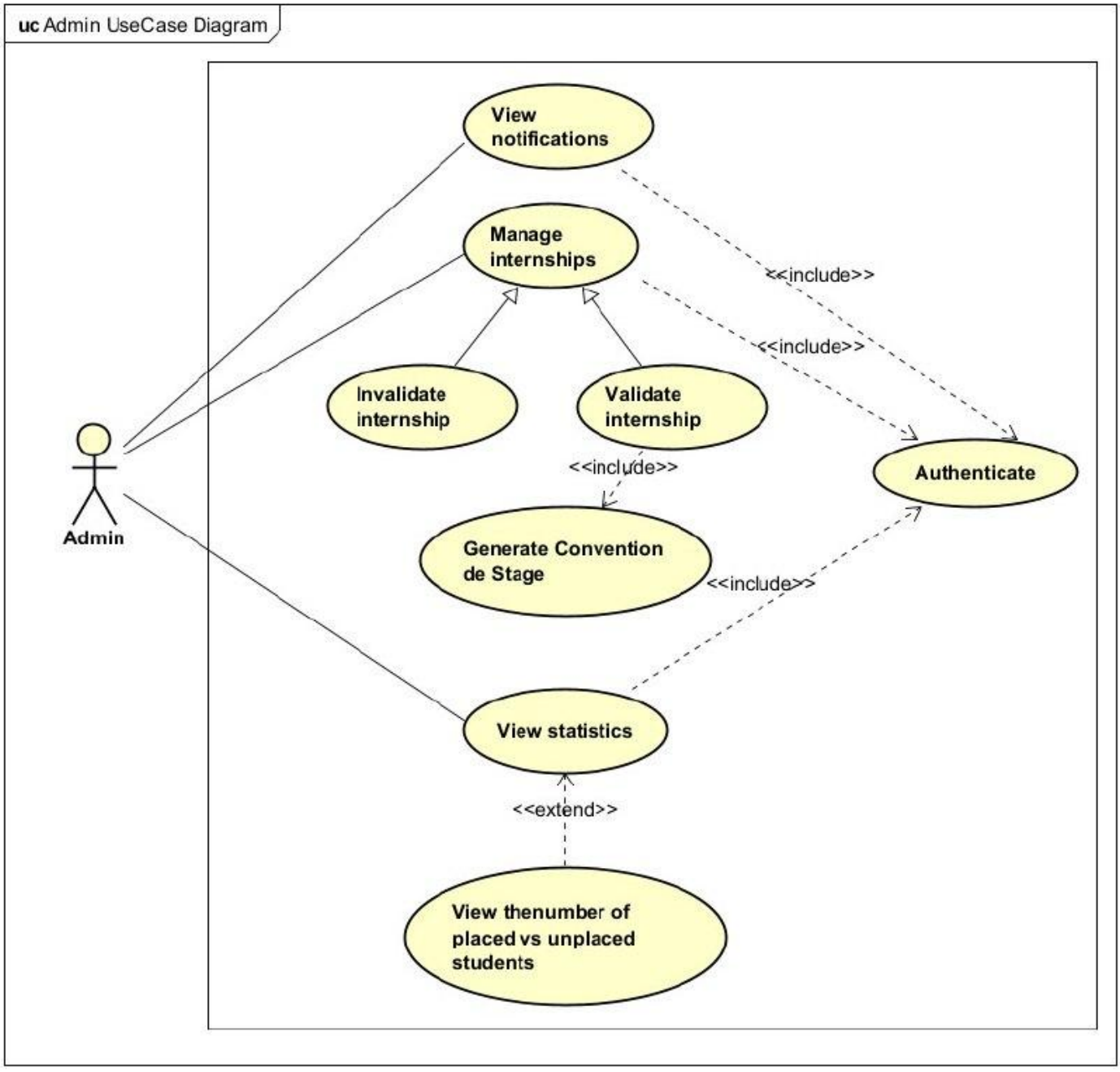


Figure 7: Admin Use Case diagram

2.6 Detailed textual description:

2.6.1 Student textual description:

Case	Apply for an internship
Primary Actor	Student
Version	1.0
Preconditions	The student must be authenticated.
Postconditions	The system records the application and notifies the recruiter.
Nominal Scenario	1. Student searches for an internship. 2. The system displays a list of available offers. 3. Student selects a specific internship offer. 4. The system displays the offer details and an "Apply" button. 5. Student clicks on the "Apply" button. 6. The system retrieves the student's personal details and technical skills. 7. Student confirms the submission of the application. 8. The system displays a success message.
Alternative Scenario	**A1: Student needs to update profile before applying.** A1 starts at point 4 of the nominal scenario. 5. Student realizes their technical skills are outdated. 6. Student navigates to "Edit personal details". 7. Student updates skills and returns to the offer. **A1 continues from point 5 of the nominal scenario**

Exceptional Scenarios	**E1: Application already submitted.** E1 starts at point 3 of the nominal scenario. 4. The system detects the student has already applied. 5. The system disables the apply button and shows a warning message. E1 continues from point 3 of the nominal scenario

Figure 8: Student applies for an internship

2.6.2 Company textual description:

Case	Create internship offer
Primary Actor	Company
Version	1.0
Preconditions	The company representative must be authenticated.
Postconditions	A new internship offer is published and visible to students.
Nominal Scenario	1. Company accesses the "Manage internship offers" section. 2. Company selects the option to "Create internship offer". 3. The system displays a form for offer details (title, description, duration, requirements). 4. Company fills in the required information. 5. Company clicks the "Publish" button. 6. The system validates the data and saves the offer. 7. The system displays a confirmation message.
Alternative Scenario	**A1: Modify an existing internship offer.** A1 starts at point 1 of the nominal scenario. 2. Company selects an existing offer from the list. 3. Company clicks "Modify".

	4. The system loads the current data into the form. 5. Company updates the fields (e.g., changing the Tech Tags). 6. Company clicks "Update". A1 continues from point 1 of the nominal scenario. **A2: Delete an existing internship offer.** A2 starts at point 1 of the nominal scenario. 2. Company selects an existing offer. 3. Company clicks "Delete". 4. The system asks for confirmation. 5. Company confirms deletion. 6. The system removes the offer from the student search results. A2 continues from point 1 of the nominal scenario.
Exceptional Scenarios	**E1: Mandatory fields are missing.** E1 starts at point 5 of the nominal scenario. 6. The system detects empty required fields. 7. The system highlights the missing fields in red. E1 continues from point 4 of the nominal scenario.

Figure 8: Company creates an internship offer

2.6.3 Admin textual description:

Case	Manage Internships
Primary Actor	Admin (Univ. Administration)
Version	1.0
Preconditions	Admin must be authenticated.

Postconditions	Internship is validated in the system and the "Convention de Stage" is generated.
Nominal Scenario	1. Admin accesses the "View Notifications" section to see new internship requests. 2. Admin selects a specific pending internship from the "Manage Internships" dashboard. 3. Admin reviews the student and company details. 4. Admin clicks on "Validate Internship". 5. The system automatically generates the "Convention de Stage" in PDF format. 6. The system pre-fills the PDF with data from the student, company, and university. 7. The system displays a success message and sends the PDF to both parties.
Alternative Scenario	**A1: Invalidate/Reject an internship request.** A1 starts at point 3 of the nominal scenario. 4. Admin selects "Invalidate Internship". 5. The system prompts for a reason for rejection. 6. Admin enters the reason and confirms. 7. The system updates the status to "Invalidated". A1 continues from point 1 of the nominal scenario.
Exceptional Scenarios	**E1: Document Generation Failure.** E1 starts at point 4 of the nominal scenario. 5. The system detects missing mandatory data (e.g., missing University address or Student ID) required for the PDF template. 6. The system cancels the generation process.

	7. The system displays an error: "Cannot generate agreement: Missing profile information." E1 continues from point 1 of the nominal scenario.

Figure 9: Admin manages internships

2.7 Conclusion

This chapter presented the preliminary study and detailed specifications for the Stag.io platform.

We began by introducing the project vision and the three main actors who will interact with the system: students, companies, and administrators. Each actor has specific roles and responsibilities within the platform.

We then defined the functional requirements, listing all the features the system must provide for each actor. These include registration, profile management, internship search, application tracking, offer management, validation workflows, and document generation.

We also defined the non-functional requirements, covering performance, security, usability, reliability, maintainability, and scalability. These ensure that the system will not only function correctly but also perform well under real-world conditions.

Together, these requirements form a complete specification that will guide the design and implementation phases of the project. The next chapter will focus on the analysis and design of the system, including class diagrams, sequence diagrams, and activity diagrams based on the requirements defined here.

Chapter 3: Analysis and Design

3.1 Introduction

After defining the functional and non-functional requirements in the previous chapter, this chapter focuses on the analysis and design of the Stag.io platform. The goal is to translate the requirements into a concrete blueprint that will guide the implementation phase. We begin by presenting the class diagram, which shows the static structure of the system including classes, attributes, and relationships. Next, we present the navigation diagram to illustrate how users will move through the application. We then model the dynamic behavior of the system using activity diagrams and sequence diagrams for three selected use cases. Finally, we present the relational mapping that shows how the classes will be transformed into database tables. Together, these diagrams provide a complete view of the system before any code is written.

3.2 System Class Diagram

A system class diagram is a type of UML (Unified Modeling Language) diagram that models the static structure of a system by showing its classes, their attributes and methods, and the relationships between them. It serves as a blueprint of the system's architecture, capturing how different entities are organized and interact at a structural (not behavioral) level.The key elements of a use case diagram include:

➢ Class**:** represents an entity in the system, divided into three sections — the class name, its attributes, and its methods.

➢ Attribute**:** represents a property or data field that describes the characteristics of a class.

➢ Method**:** represents an operation or function that a class can perform.

➢ Association**:** represents a general structural link or connection between two classes.

➢ Aggregation**:** represents a "has-a" relationship where the child class can exist independently of the parent.

➢ Composition**:** represents a stronger "has-a" relationship where the child class cannot exist without the parent.

➢ Inheritance**:** represents an "is-a" relationship where a subclass extends and inherits from a parent class.

➢ Dependency**:** represents a loose relationship where one class temporarily uses or depends on another.

➢ Multiplicity**:** represents the number of instances allowed in a relationship between classes.

3.2.1 Stag.io System Class diagram:

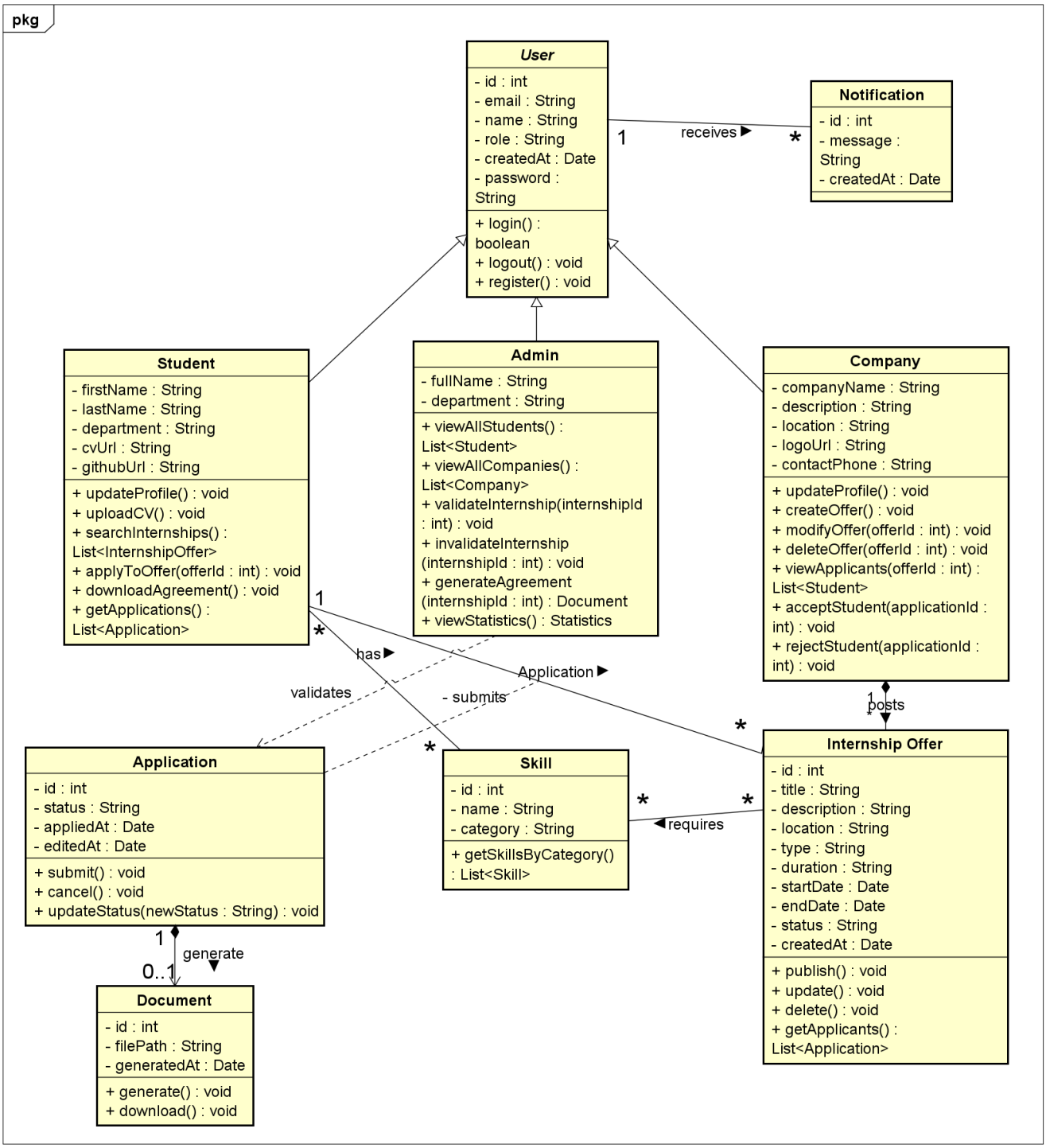


Figure 10: Stag.io System Class diagram

3.3 Navigation Diagram

A navigation activity diagram is a UML diagram that models the flow of navigation through a system's pages and screens, showing how users move from one interface to another based on actions and conditions.

➢ Initial Node: represents the starting point of the navigation flow, shown as a filled black circle.

➢ Final Node: represents the end point of the navigation flow, shown as a filled black circle inside a larger circle.

➢ Page: represents a full navigable screen in the system, shown as a rounded rectangle labeled with <<page>>.

➢ Frame: represents a partial screen or embedded component within a page, shown as a rounded rectangle labeled with <<frame>>.

➢ Decision Node: represents a branching point where the flow splits based on a condition, shown as a diamond shape.

➢ Transition: represents the navigation flow from one page or node to another, shown as a directed arrow between elements.

➢ Guard Condition: represents a condition that must be met for a specific transition to occur, shown in square brackets on a transition arrow (e.g., [valid login]).

3.3.1 Student Navigation diagram:

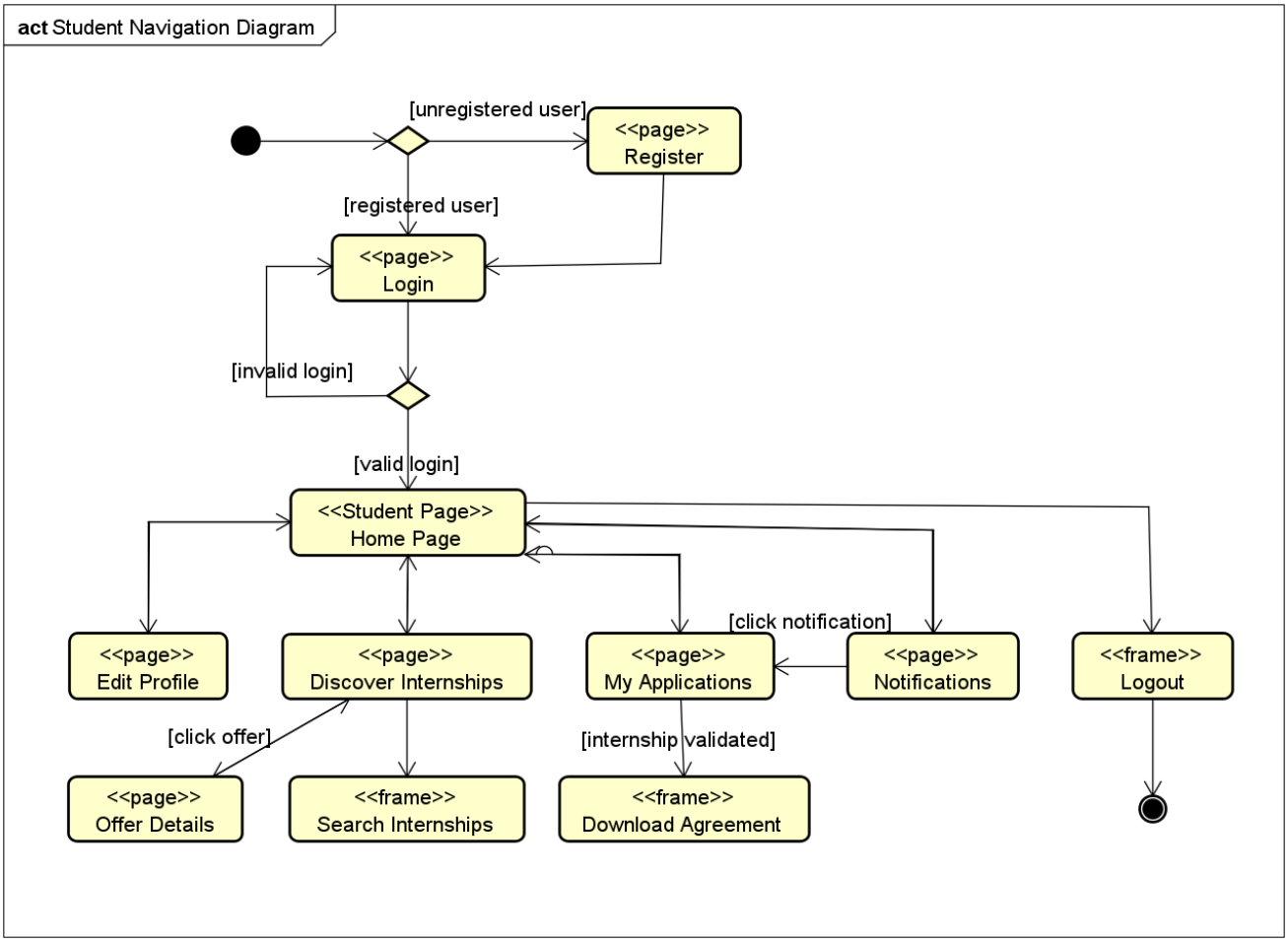


Figure 11: Student Navigation diagram

3.3.2 Company Navigation diagram:

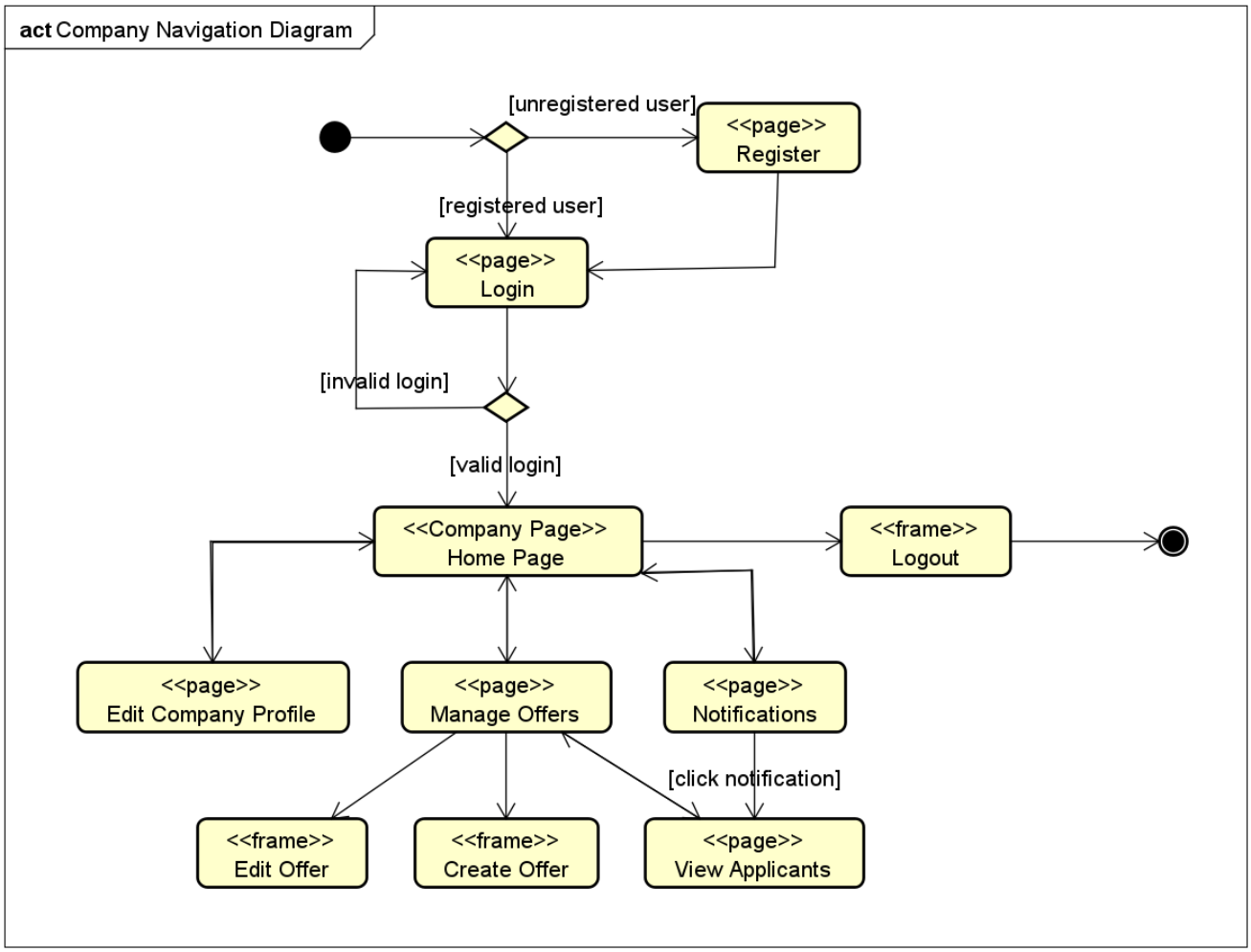


Figure 12: Company Navigation diagram

3.3.3 Admin Navigation diagram:

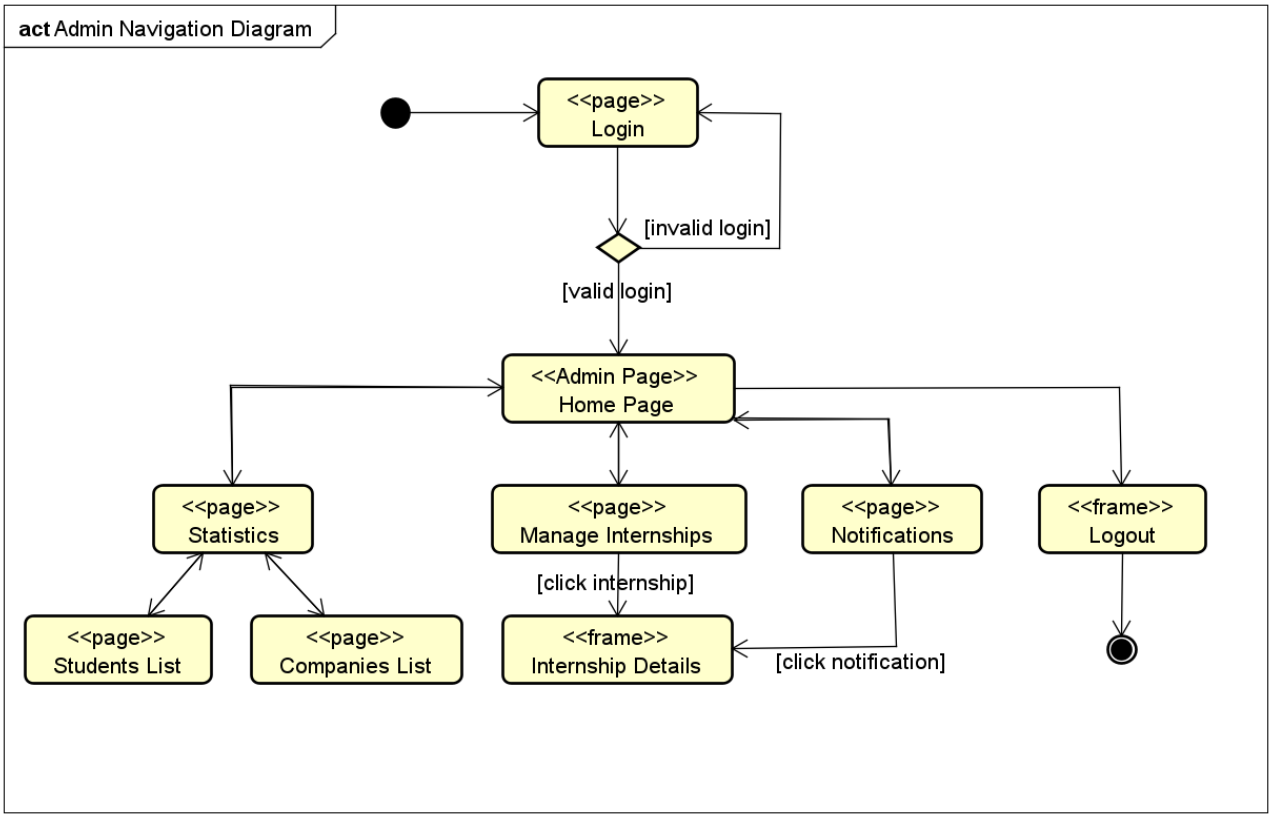


Figure 13: Admin Navigation diagram

3.4 Activity Diagram

An activity diagram is a UML diagram that models the dynamic flow of actions and decisions within a system, showing the sequence of activities performed to complete a process or use case. The key elements of a use case diagram include:

➢ Initial Node: represents the starting point of the activity flow, shown as a filled black circle.

➢ Final Node: represents the end point of the activity flow, shown as a filled black circle inside a larger circle.

➢ Action: represents a single step or task performed within the process, shown as a rounded rectangle.

➢ Decision Node: represents a branching point where the flow splits into alternative paths based on a condition, shown as a diamond shape.

➢ Merge Node: represents the point where multiple alternative paths converge back into a single flow, shown as a diamond shape.

➢ Fork Node: represents a point where a single flow splits into multiple parallel flows, shown as a thick horizontal or vertical bar.

➢ Join Node: represents a point where multiple parallel flows synchronize back into one, shown as a thick horizontal or vertical bar.

➢ Transition: represents the flow of control from one action or node to the next, shown as a directed arrow.

➢ Guard Condition: represents a condition that controls which path is taken at a decision node, shown in square brackets on a transition arrow (e.g., [valid login]).

➢ Swimlane: represents the responsibility partition that groups actions by the actor or system component performing them, shown as a labeled column or row.

3.4.1 Activity diagram for ‘Apply for an Internship’ case:

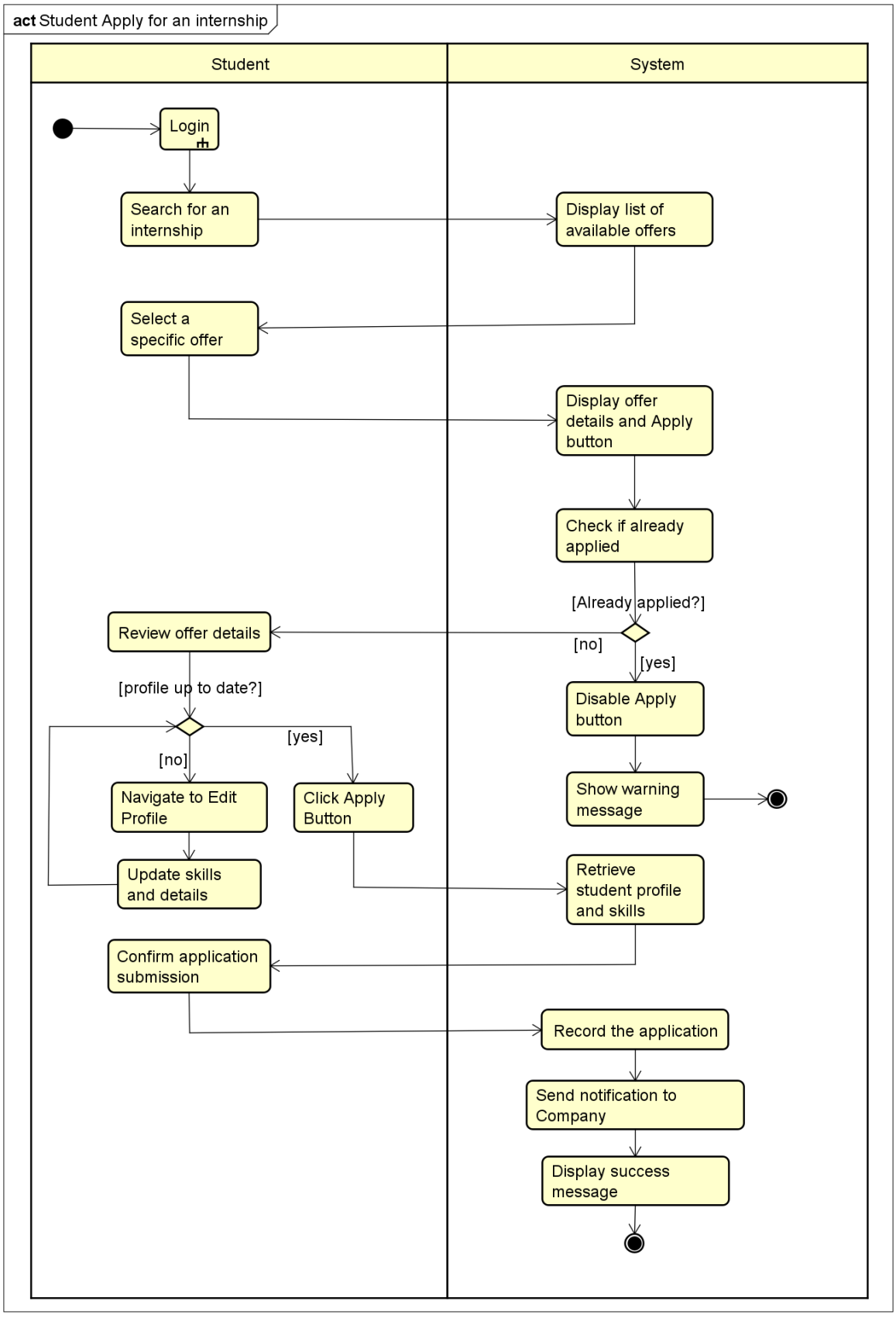


Figure 14: Activity Diagram for ‘Apply for an Internship’ Case

3.4.2 Activity diagram for ‘Create Internship Offer’ case:

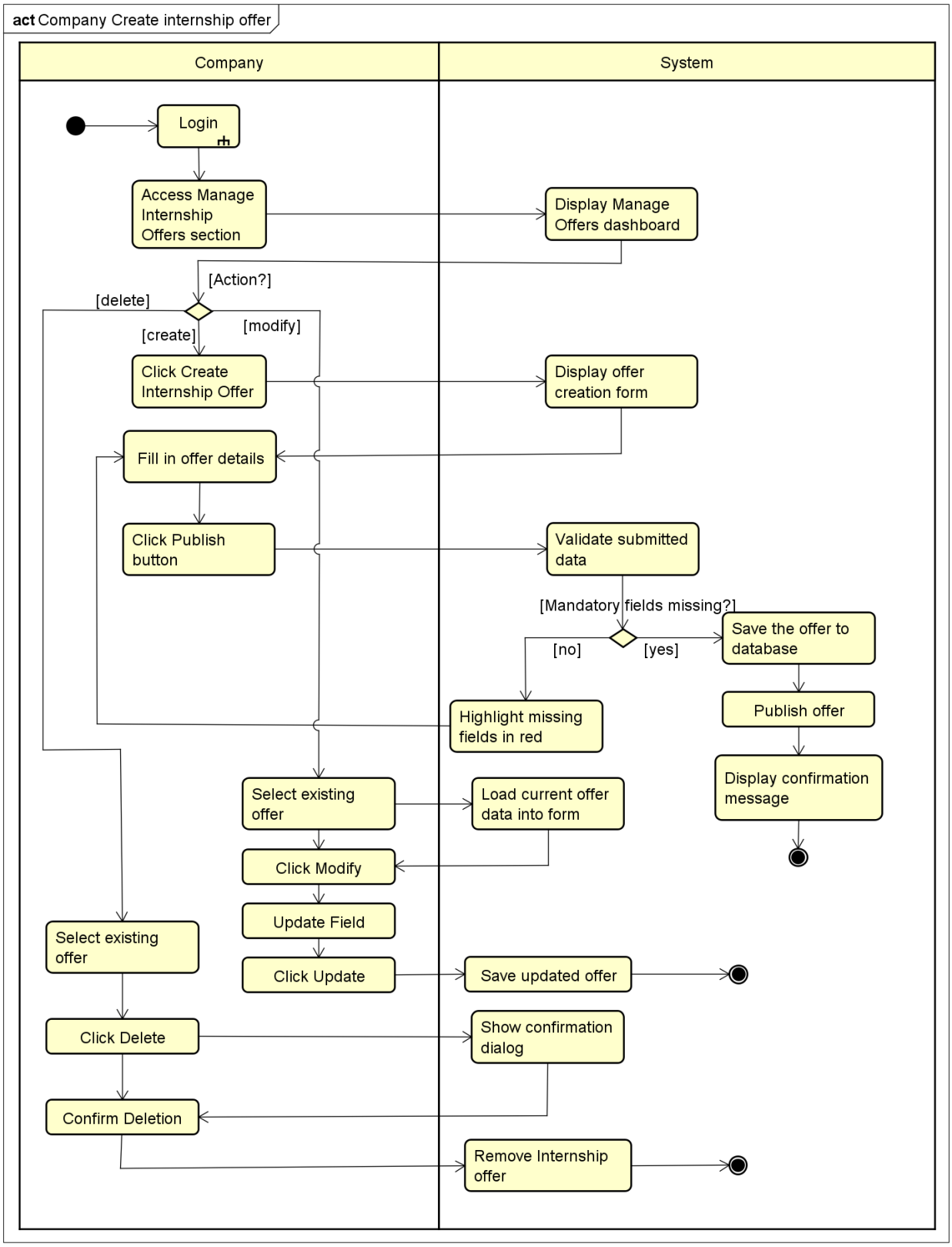


Figure 15: Activity Diagram for ‘Create Internship Offer’ Case

3.4.3 Activity diagram for ‘Manage Internships’ case:

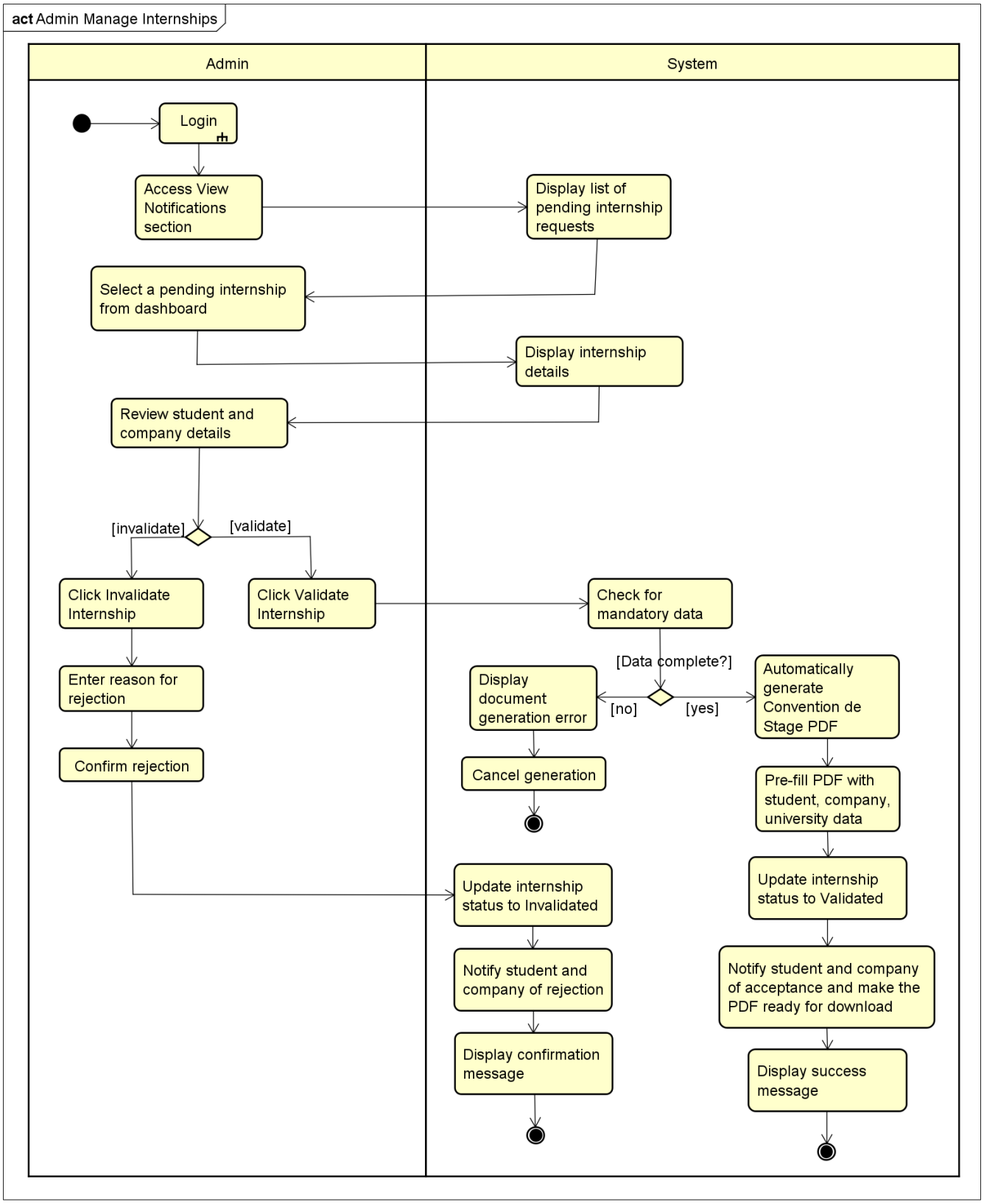


Figure 16: Activity Diagram for ‘Manage Internships’ Case

3.5 Sequence Diagram

A sequence diagram is a UML diagram that models the dynamic interaction between objects or components in a system, showing the order in which messages are exchanged to complete a specific process or use case.

➢ Actor: represents an external user or system that initiates or participates in the interaction, shown as a stick figure.

➢ Object/Participant: represents a class instance or system component involved in the interaction, shown as a labeled rectangle at the top of the diagram.

➢ Lifeline: represents the existence of an object over time during the interaction, shown as a dashed vertical line extending downward from each participant.

➢ Activation Bar: represents the period during which an object is active and executing an operation, shown as a thin rectangle on the lifeline.

➢ Message: represents a communication or call sent from one participant to another, shown as a solid horizontal arrow between lifelines.

➢ Return Message: represents the response sent back after a message has been processed, shown as a dashed horizontal arrow pointing back to the caller.

➢ Self-Message: represents a call that an object sends to itself, shown as an arrow that loops back to the same lifeline.

➢ Guard Condition: represents a condition that must be true for a message to be sent, shown in square brackets above the message arrow (e.g., [valid login]).

➢ Loop Fragment: represents a repeated interaction that occurs multiple times based on a condition, shown as a labeled frame with the keyword "loop".

➢ Alt Fragment: represents alternative interaction paths based on a condition, shown as a labeled frame divided by a dashed line with the keyword "alt".

3.5.1 Sequence diagram for ‘Apply for an Internship’ case:

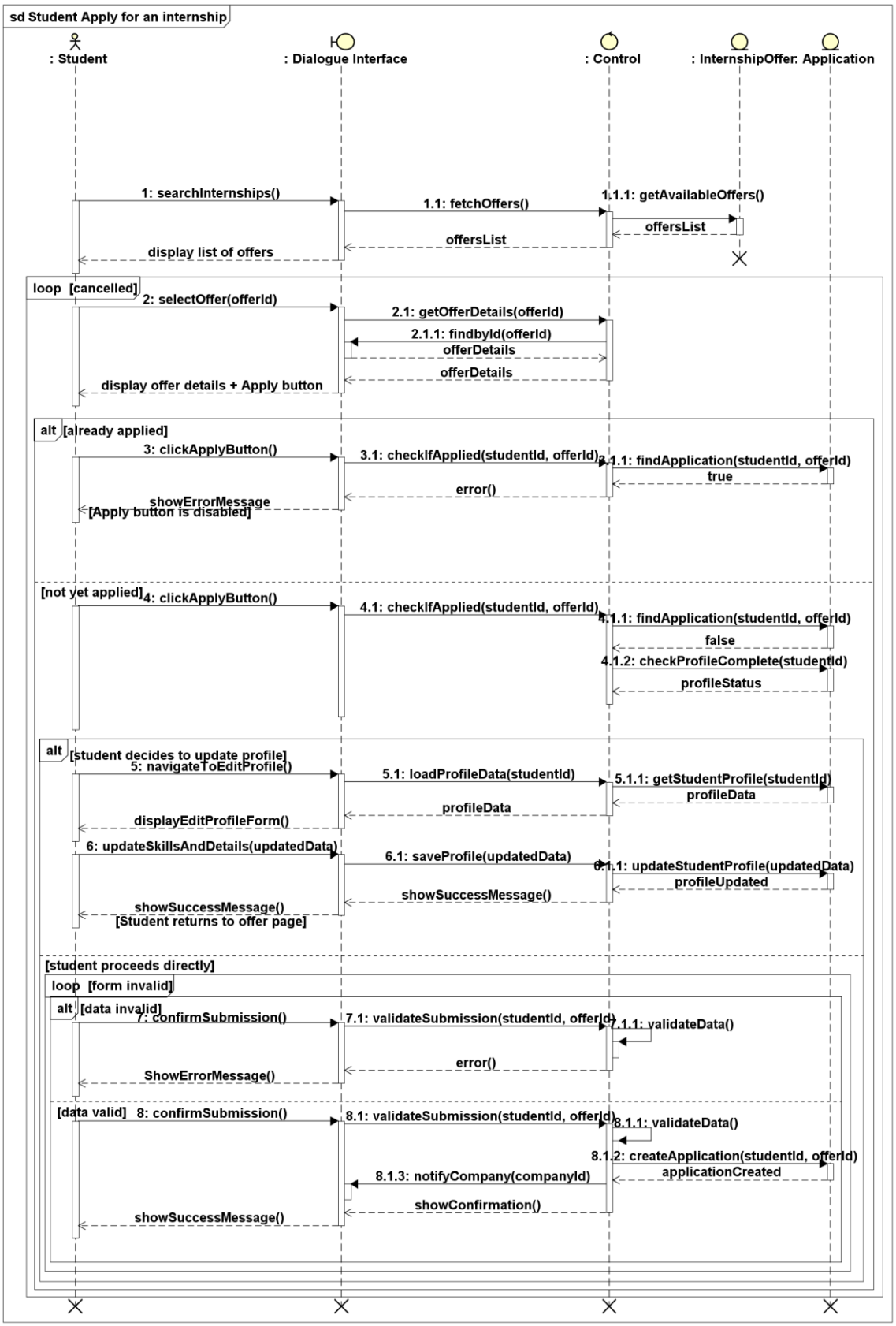


Figure 17: Sequence Diagram for ‘Apply for an Internship’ Case

3.5.2 Sequence diagram for ‘Create Internship Offer’ case:

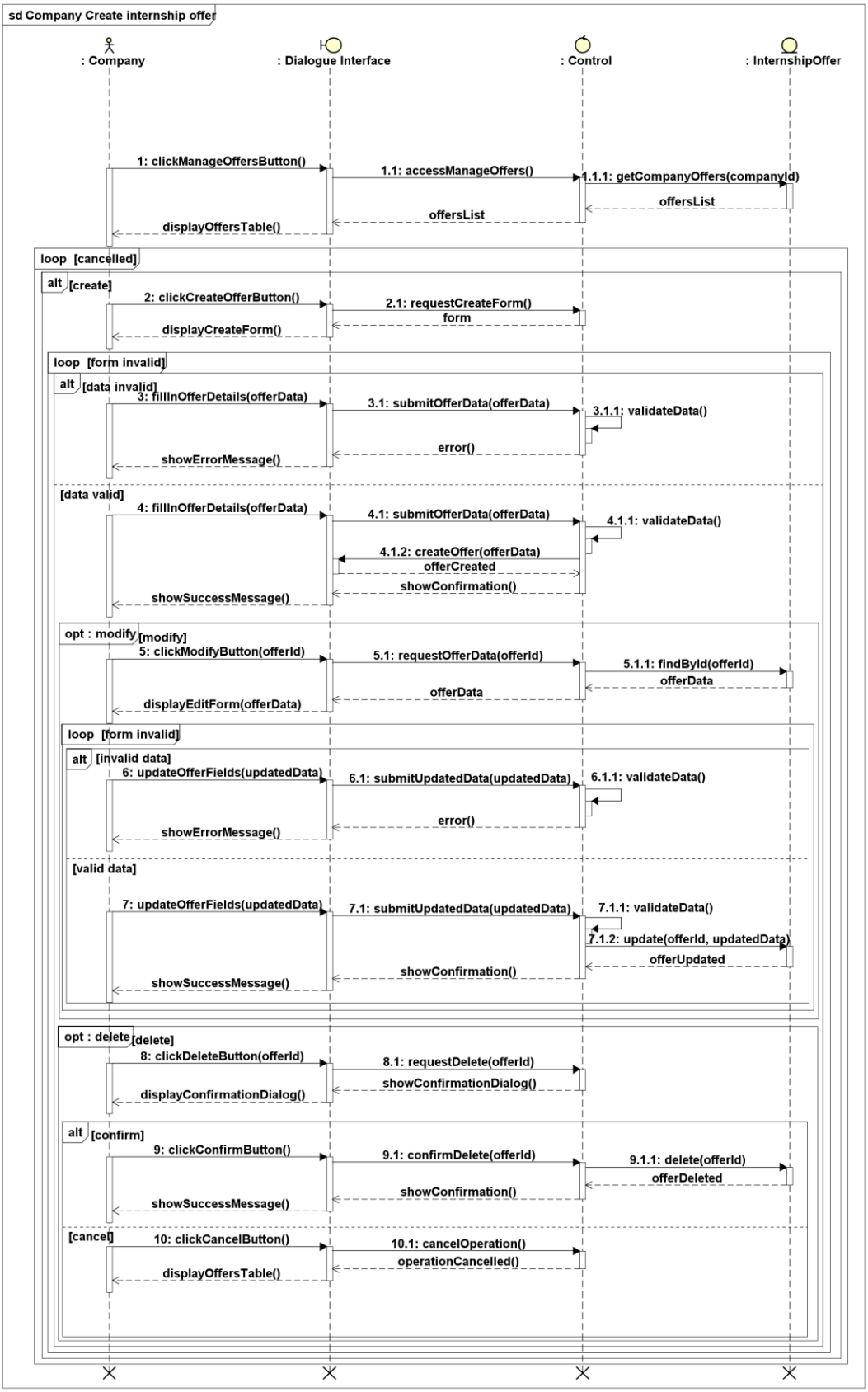


Figure 18: Sequence Diagram for ‘Create Internship Offer’ Case

3.5.3 Sequence diagram for ‘Manage Internships’ case:

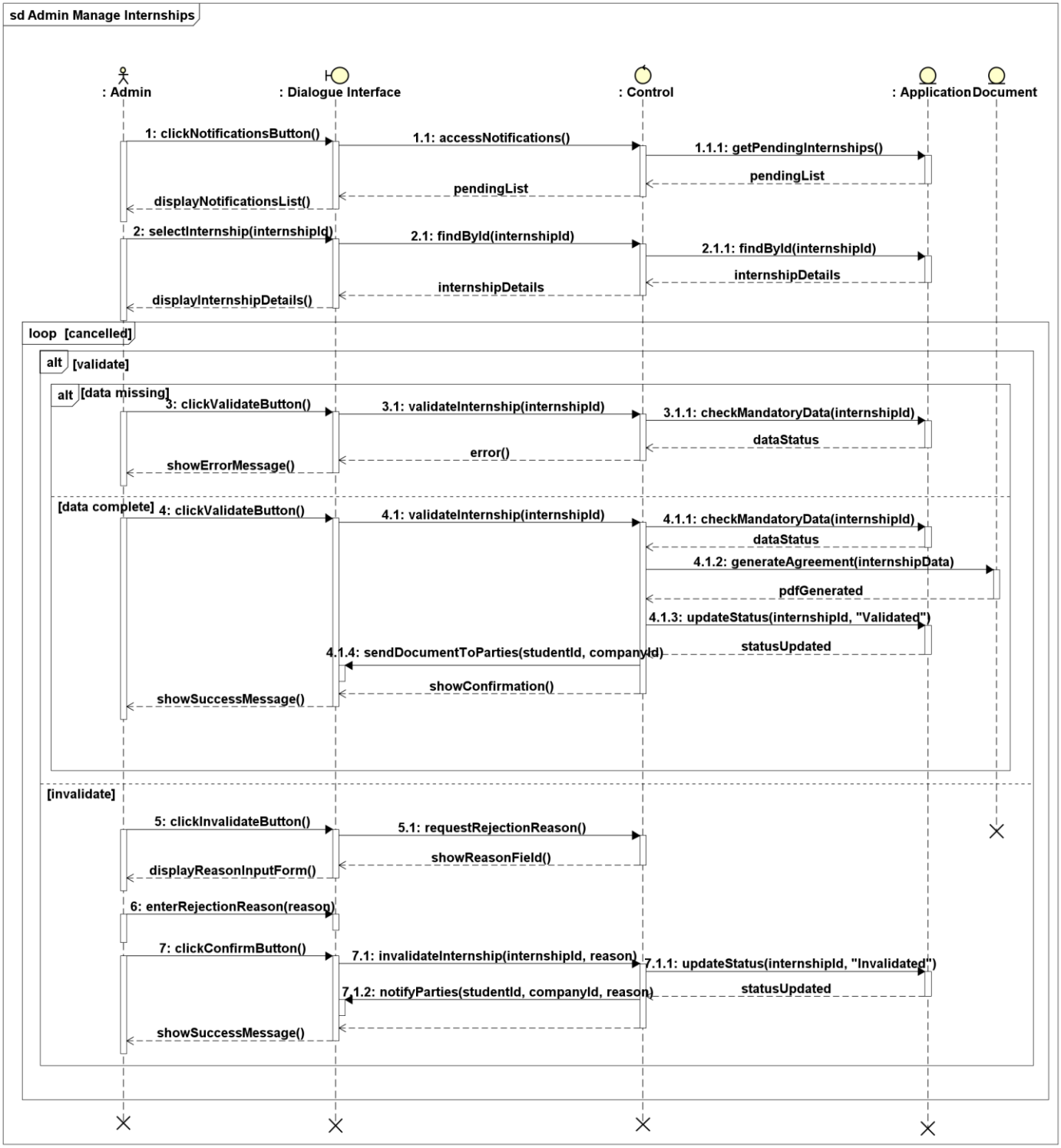


Figure 19: Sequence Diagram for ‘Manage Internships’ Case

3.6 Transition to Relational Modeling From the Class Diagram

The transition to relational modeling is the process of converting a UML class diagram into a relational database schema, where classes become tables, attributes become columns, and relationships are represented using primary and foreign keys.

➢ Table: represents an entity from the class diagram converted into a database table that stores records of that entity.

➢ Primary Key (PK): represents a unique identifier assigned to each record in a table to distinguish it from others.

➢ Foreign Key (FK): represents a column that references the primary key of another table to establish a relationship between them.

➢ Column: represents an attribute from the class diagram converted into a field within a database table.

➢ One-to-One Relationship: represents an association where one record in a table corresponds to exactly one record in another table.

➢ One-to-Many Relationship: represents an association where one record in a table corresponds to multiple records in another table.

➢ Many-to-Many Relationship: represents an association where multiple records in one table correspond to multiple records in another, resolved using a junction table.

➢ Junction Table: represents an intermediary table created to resolve a many-to-many relationship by holding the foreign keys of both related tables.

➢ Inheritance Mapping: represents the conversion of an inheritance relationship into the relational model by merging or linking subclass attributes to the parent table.

3.6.1 Relational Model:

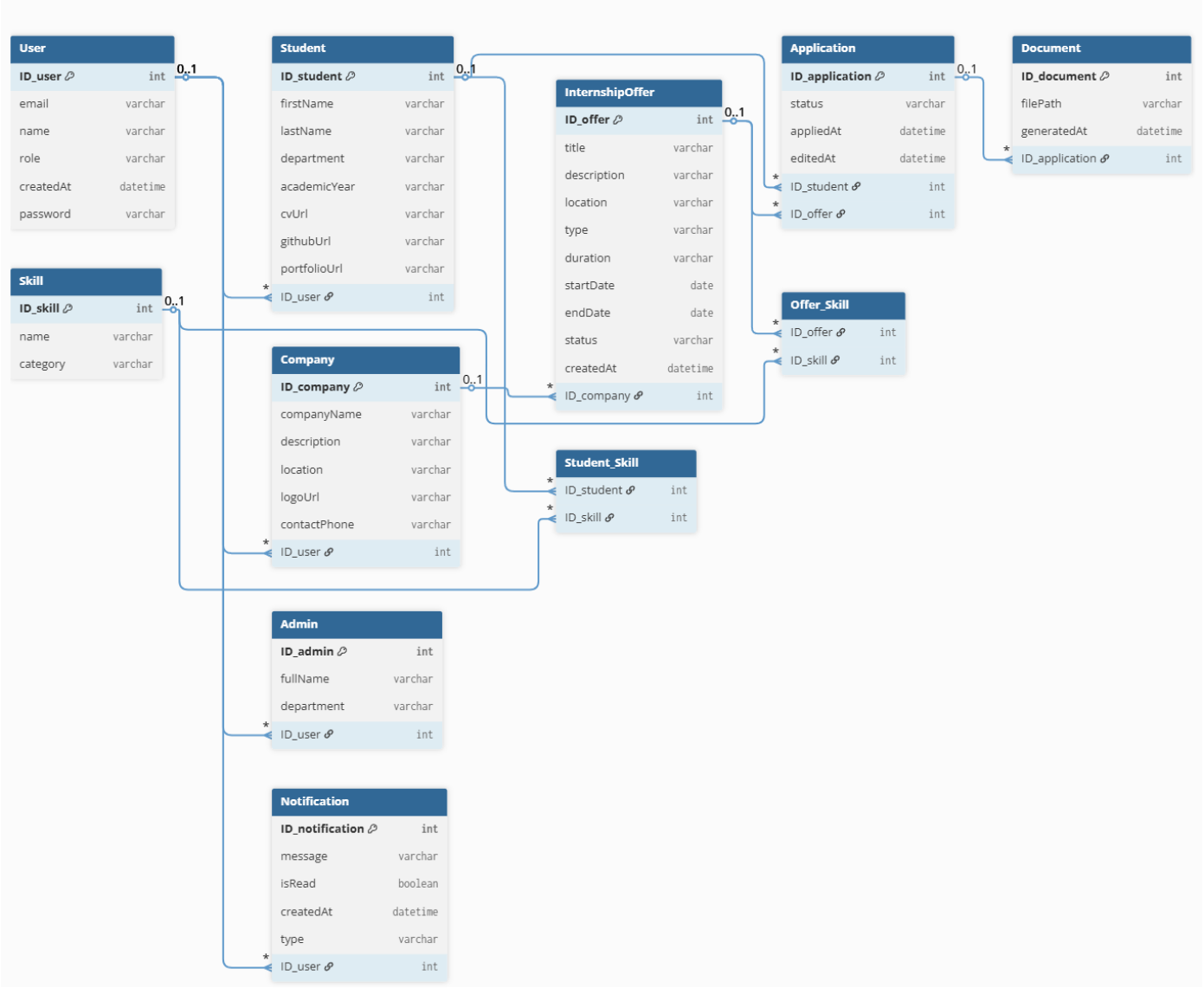


Figure 20: Relational Model Diagram

3.6.2 Relationships:

➢ User(ID_user, email, name, role, createdAt, password)

➢ Student(ID_student, firstName, lastName, department, academicYear, cvUrl, githubUrl, portfolioUrl, ID_user*)

➢ Company(ID_company, companyName, description, location, logoUrl, contactPhone, ID_user*)

➢ Admin(ID_admin, fullName, department, ID_user*)

➢ Skill(ID_skill, name, category)

➢ InternshipOffer(ID_offer, title, description, location, type, duration, startDate, endDate, status, createdAt, ID_company*)

➢ Application(ID_application, status, appliedAt, editedAt, ID_student*, ID_offer*)

➢ Document(ID_document, filePath, generatedAt, ID_application*)

➢ Notification(ID_notification, message, isRead, createdAt, type, ID_user*)

➢ Student_Skill(ID_student*, ID_skill*)

➢ Offer_Skill(ID_offer*, ID_skill*)

3.7 Conclusion

This chapter presented the complete analysis and design of the Stag.io platform, covering both its static structure and dynamic behavior.

We modeled the system through a series of UML diagrams, including the class diagram, navigation diagrams, activity diagrams, and sequence diagrams, each addressing a different aspect of the system for all three actors: Student, Company, and Admin.

Finally, we derived the relational database schema from the class diagram, producing a structured and consistent data model ready for implementation.

Together, these diagrams form a solid foundation that will guide the implementation phase of the project.

Chapter 4: Implementation

4.1 Introduction

This chapter presents the implementation of the Stag.io platform, moving from the analysis and design artifacts produced in Chapter 3 to a working full-stack web application. The chapter covers the proposed system architecture, the hardware and software resources used during development, selected code and data-handling examples, the database structure, and a walkthrough of the main user interfaces across all three actor spaces: Student, Company, and Administrator.

4.2 Proposed System Architecture

Stag.io follows a modern three-tier client-server architecture consisting of a presentation layer (frontend), a business logic layer (backend API), and a data layer (database). The three layers communicate over HTTP using RESTful conventions, and all data is exchanged in JSON format. The Frontend layer is built with React.js, which runs entirely in the browser, sends HTTP requests to the backend API, receives JSON responses, and renders the data for all three actor dashboards (Student, Company, and Administration). The Backend layer is built with Python and Django REST Framework, exposing a RESTful API that handles authentication, enforces business rules, and communicates with the database; all endpoints are documented and testable via Swagger UI at /api/docs/. The Database layer uses PostgreSQL 13, where Django's ORM manages all migrations, queries, and table relationships automatically. Finally, authentication is handled through JSON Web Tokens (JWT): the access token is valid for 30 minutes, the refresh token lasts 7 days, and every protected endpoint verifies the token and identifies the user's role before processing the request.

4.3 The Hardware and Software Resources Used For the Realization of the Project

4.3.1 Hardware specifications:

The following machines were used during the development and testing of the Stag.io platform:

Component	Specification
Manufacturer	Hp
Processor	AMD Ryzen 5 PRO 5650U @2.3GHz 6 cores 12 threads with Radeon Graphics
Ram	16.00 GB
Storage	500 GB

Figure 21: Laptop 1 Specifications

Component	Specification
Manufacturer	Dell
Processor	Intel Core i5-8365U de 8th gen 4 cores/8 threads, 1,6 GHz à 4,1 GHz
Ram	16.00 GB
Storage	500 GB

Figure 22: Laptop 2 Specifications

Component	Specification
Manufacturer	Hp
Processor	Intel Core i7-8665U 8th gen 4 cores/8 threads, 1,9 GHz à 4,8 GHz
Ram	16.00 GB
Storage	1000 GB

Figure 23: Laptop 3 Specifications

The web server machine used:

Component	Specification
Test Interfaces	Google Chrome 147 (localhost:8000) and Brave Realease (localhost:8000)

API Testing Tool	Swagger UI (drf-spectacular)
Local Server	Django development server (WSGIServer/0.2)
Network	Localhost (127.0.0.1)

Figure 23: Testing Environment

4.3.2 Versions of the software tools used in the development of the project:

Stag.io application is powered by a stack consisting of Django, React and PostgreSQL technologies. The backend was implemented using Django 3.2, a robust web framework that utilizes Python 3.11 programming language and offers great out-of-the-box functionality such as advanced ORM capabilities, automatic migration mechanisms, enhanced security settings, and easy implementation with Django REST Framework to develop REST APIs. Frontend is implemented with React.js 18, a component-oriented JavaScript framework created by Meta company which uses a virtual DOM to guarantee fast rendering operations, an effective state management system across the three actor dashboards, and a big variety of available libraries. The database is represented by PostgreSQL 13, an ACID-compliant, open-source relational database system with sophisticated query capabilities and convenient connection with Django via psycopg2 connector, making it ideal for the platform's multi-table inheritance structure and many-to-many relationship of skills among actors. Thus, the above mentioned architecture of the platform allows implementing a clean separation of concerns between data, business logic and presentation layer.



Figure 24: Django Logo

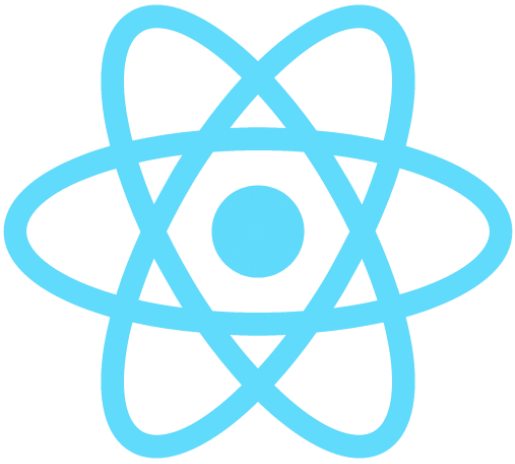


Figure 25: React Logo

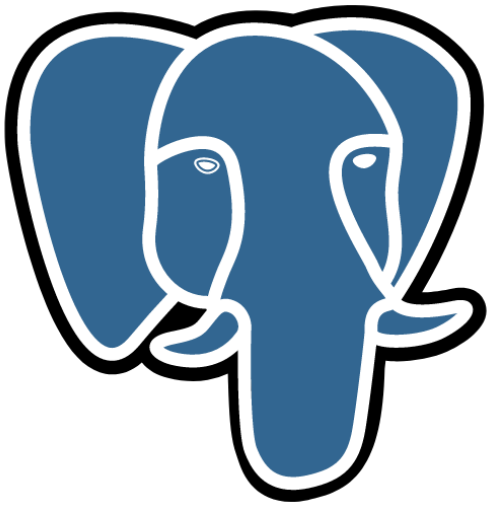


Figure 26: PostgreSQL Logo

The following software tools, frameworks, and libraries were used to develop the Stag.io platform:

Tool / Library	Version	Purpose
Python	3.11.9	Core programming language for backend
Django	3.2.25	Web framework — routing, ORM, migrations
Django REST Framework	3.12.4	RESTful API layer — serializers, views, authentication
Simple JWT	4.8.0	JWT-based authentication (access + refresh tokens)
drf-spectacular	0.15.1	Automatic Swagger UI / OpenAPI 3 documentation
psycopg2-binary	2.9.x	PostgreSQL database driver for Python
Pillow	9.2+	Image processing for profile photos and logos
WeasyPrint	latest	PDF generation for internship agreements (Convention de Stage)
PostgreSQL	13	Relational database management system

Figure 27: Backend Software Tools

Tool / Library	Version	Purpose
React.js	18.x	UI component library for building the web interface
Vite	5.x	Frontend build tool and development server

React Router	6.x	Client-side routing between pages
Axios	1.x	HTTP client for communicating with the Django API
Tailwind CSS	3.x	Utility-first CSS framework for responsive design

Figure 28: Frontend Software Tools

Tool / Library	Version	Purpose
VS Code	1.107	Primary code editor
Git	2.x	Version control — project hosted on GitHub
pgAdmin 4	Latest	PostgreSQL database GUI management
Google Chrome	147	Frontend testing browser
GTK3 Runtime	3.x	Windows dependency for WeasyPrint PDF generation

Figure 29: Development Tools

Astah UML: Astah UML is a modeling tool that supports UML (Unified Modeling Language) and mind maps. Designed to offer all the necessary features for UML modeling without excessive complexity, Astah UML is simple to learn and easy to use for creating the required UML diagrams. Perfect for modeling all the necessary UML diagrams for Stag.io.

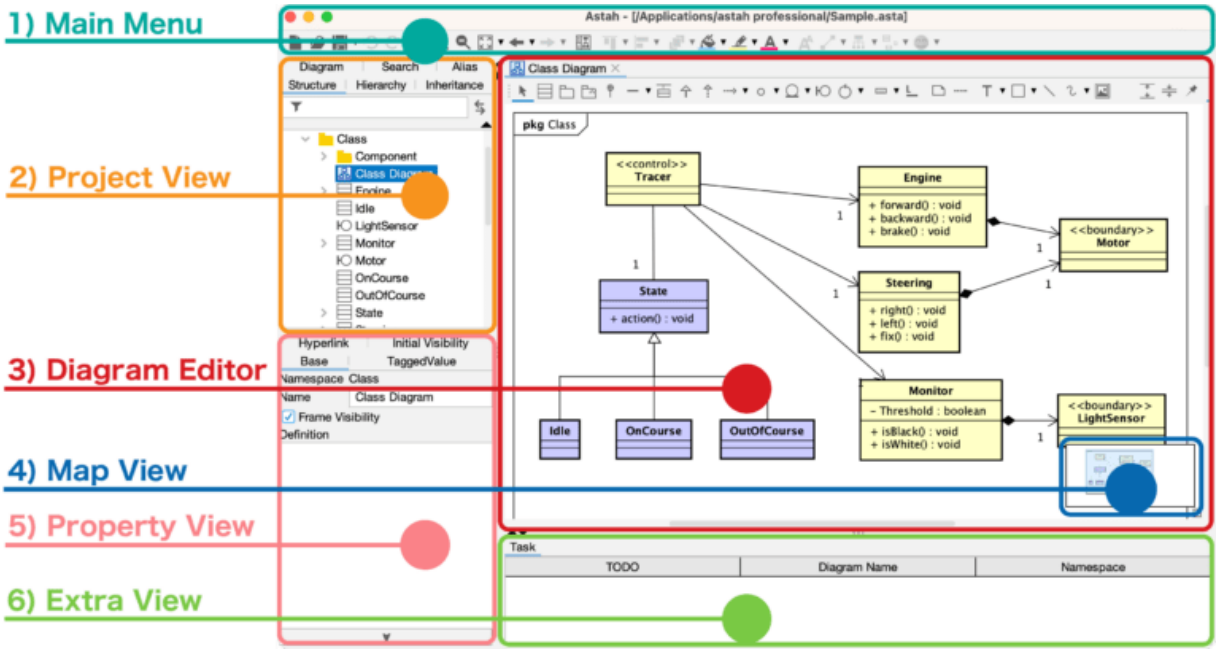


Figure 30: AstahUML Interface

4.4 Some examples of data and processing implementation

This section presents selected code excerpts that illustrate the core data structures and business logic implemented in the Stag.io backend. Each example is accompanied by an explanation of its role in the system.

4.4.1 JWT Authentication and Role-Based Permissions:

All API endpoints are protected by JWT authentication. The backend defines three custom permission classes IsStudent, IsCompany, and IsAdmin that check the role field on the authenticated user. This ensures that a student cannot access company endpoints and vice versa. The token is extracted from the Authorization header on every request by Django REST Framework's JWTAuthentication class.

```python
class IsAdmin(permissions.BasePermission):
    """Allow access only to Django staff admins (is_staff=True)."""

    def has_permission(self, request, view):
        return (
            request.user
            and request.user.is_authenticated
            and request.user.is_staff
        )


class IsStudent(permissions.BasePermission):
    """Allow access only to student users."""

    def has_permission(self, request, view):
        return (
            request.user
            and request.user.is_authenticated
            and getattr(request.user, 'role', '') == 'student'
        )


class IsCompany(permissions.BasePermission):
    """Allow access only to company users."""

    def has_permission(self, request, view):
        return (
            request.user
            and request.user.is_authenticated
            and getattr(request.user, 'role', '') == 'company'
        )
```

Figure 31: Permission classes for role-based access control (user/views.py)

4.4.2 InternshipOffer Model with Computed Duration:

The InternshipOffer model represents a job posting created by a company. Beyond its standard database fields, it includes a Python @property called duration_display that computes a human-readable duration string from the start and end dates at runtime, without storing it in the database. The logic applies sensible rounding: values close to a month boundary are shown as months, values close to a week boundary are shown as weeks, and all other values are shown as exact days.

```python
class InternshipOffer(models.Model):
    """
    An internship offer posted by a Company.
    Students will browse and apply to these offers.
    """
    class Type(models.TextChoices):
        PAID = 'paid', 'Paid'
        UNPAID = 'unpaid', 'Unpaid'

    class Status(models.TextChoices):
        OPEN = 'open', 'Open'
        CLOSED = 'closed', 'Closed'

    company = models.ForeignKey( Company, on_delete=models.CASCADE, related_name='offers',)
    title = models.CharField(max_length=255), description = models.TextField(), location = models.CharField(max_length=255)
    wilaya = models.CharField(max_length=100, choices=ALGERIAN_WILAYAS, blank=True)
    is_remote = models.BooleanField( default=False, help_text='If True, this is a remote/online internship (no location or wilaya).',)
    type = models.CharField( max_length=10, choices=Type.choices, default=Type.UNPAID,)
    salary_per_week = models.DecimalField( max_digits=10, decimal_places=2, null=True, blank=True, )
    start_date = models.DateField()
    end_date = models.DateField()
    photo = models.ImageField( null=True, blank=True, upload_to='uploads/offer_photos/', )
    status = models.CharField( max_length=10, choices=Status.choices, default=Status.OPEN, )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

```python
    @property
    def duration_display(self):
        """
        Human-readable duration with sensible rounding.
        - Within 5 days of a month → 'X months'
        - Within 3 days of a week  → 'X weeks'
        - Otherwise                → 'X days'
        """
        delta = (self.end_date - self.start_date).days
        if delta <= 0:
            return 'Invalid dates'

        months = round(delta / 30)
        if months >= 1 and abs(delta - months * 30) <= 5:
            return f'{months} month{"s" if months > 1 else ""}'

        weeks = round(delta / 7)
        if weeks >= 1 and abs(delta - weeks * 7) <= 3:
            return f'{weeks} week{"s" if weeks > 1 else ""}'

        return f'{delta} day{"s" if delta > 1 else ""}'
```

Figure 32: InternshipOffer model with duration_display property (core/models.py)

4.4.3 Offer Search and Filtering:

The student offer search endpoint (GET /api/student/offers/) supports multiple simultaneous filters applied to the same QuerySet using Django's ORM. Filters include a text search across title and description using icontains (case-insensitive contains), an exact wilaya match, a type filter (paid/unpaid), a skills filter that requires selected skills to be present, and a duration range filter computed from start and end dates using Django's ExpressionWrapper and DurationField. Results can be ordered by most recently posted (default), highest salary, or soonest start date.



```python
class StudentOfferListView(generics.ListAPIView):
    def get_queryset(self):
        duration = params.get('duration')
        if duration == 'short':
            # 2 to 4 weeks (14 to 28 days)
            from django.db.models import ExpressionWrapper, F, DurationField
            from datetime import timedelta
            queryset = queryset.annotate(
                day_count=ExpressionWrapper(
                    F('end_date') - F('start_date'),
                    output_field=DurationField(),)
            ).filter(
                day_count__gte=timedelta(days=14),
                day_count__lte=timedelta(days=28),)
        elif duration == 'medium':
            # 1 to 2 months (29 to 60 days)
            from django.db.models import ExpressionWrapper, F, DurationField
            from datetime import timedelta
            queryset = queryset.annotate(
                day_count=ExpressionWrapper(
                    F('end_date') - F('start_date'),
                    output_field=DurationField(),)
            ).filter(
                day_count__gte=timedelta(days=29),
                day_count__lte=timedelta(days=60),)
        elif duration == 'long':
            # More than 2 months (60+ days)
            from django.db.models import ExpressionWrapper, F, DurationField
            from datetime import timedelta
            queryset = queryset.annotate(
                day_count=ExpressionWrapper(
                    F('end_date') - F('start_date'),
                    output_field=DurationField(),)
            ).filter(day_count__gt=timedelta(days=60))
```

```python
class StudentOfferListView(generics.ListAPIView):

    serializer_class = StudentOfferListSerializer
    authentication_classes = [JWTAuthentication]
    permission_classes = [IsStudent]

    def get_queryset(self):
        # Start with only open offers
        queryset = InternshipOffer.objects.filter(
            status=InternshipOffer.Status.OPEN,
        ).prefetch_related('offer_skills__skill').select_related('company')

        params = self.request.query_params

        # — Search bar ——————————————————————————
        search = params.get('search')
        if search:
            from django.db.models import Q
            queryset = queryset.filter(
                Q(title__icontains=search) | Q(description__icontains=search)
            )

        # — Wilaya filter ———————————————————————
        wilaya = params.get('wilaya')
        if wilaya:
            queryset = queryset.filter(wilaya=wilaya)

        # — Remote filter ———————————————————————
        remote = params.get('remote', '').lower()
        if remote == 'true':
            queryset = queryset.filter(is_remote=True)

        # — Skills filter (chips) ———————————————
        # Accepts comma-separated IDs: ?skills=1,3,5
```

```python
class StudentOfferListView(generics.ListAPIView):
    def get_queryset(self):

        # — Ordering ————————————————————————————
        ordering = params.get('ordering', 'recent')
        if ordering == 'salary_high':
            queryset = queryset.order_by('-salary_per_week')
        elif ordering == 'start_soon':
            queryset = queryset.order_by('start_date')
        else:
            # Default: most recently posted first
            queryset = queryset.order_by('-created_at')

        return queryset.distinct()
```

```python
# — Skills filter (chips) ———————————————
# Accepts comma-separated IDs: ?skills=1,3,5
# Returns offers that have ALL of the selected skills
skills_param = params.get('skills')
if skills_param:
    skill_ids = [
        s.strip() for s in skills_param.split(',') if s.strip().isdigit()
    ]
    for skill_id in skill_ids:
        queryset = queryset.filter(offer_skills__skill_id=skill_id)
```

Figure 33: Offer search and filtering logic (student/views.py)

4.4.4 Notification Service:

Notifications are centralized in core/notifications.py, a utility module that is called from multiple apps whenever a significant event occurs. This design avoids duplicating notification logic across views. When a company accepts a student, for example, three functions are called:

one to notify the student, one to notify all university admins, and later when the admin validates, two more functions are called to notify the student that the agreement is ready and to notify the company. Each notification stores a link field that the frontend uses to navigate the user directly to the relevant page.

```
"""
Centralized notification service.
All notification creation goes through here so the logic
is in one place and easy to maintain.
"""
from core.models import Notification, Admin

def notify(recipient, internship, notification_type, message, link=''):···

# ── Student notifications ──────────────────────────────────────

def notify_student_application_submitted(internship):···
def notify_student_accepted(internship):···
def notify_student_rejected(internship):···
def notify_student_validated(internship):···


# ── Company notifications ──────────────────────────────────────

def notify_company_new_applicant(internship):···
def notify_company_agreement_ready(internship):···
def notify_company_admin_rejected(internship):···

# ── Admin notifications ────────────────────────────────────────

def notify_admins_internship_accepted(internship):···
```

Figure 34: Centralized notification service (core/notifications.py)

4.4.5 PDF Agreement Generation:

When an admin validates an internship, the system automatically generates the official Convention de Stage (Internship Agreement) PDF using the WeasyPrint library. The generation logic is in administration/services.py. It retrieves all required data from the database (student details, company details, university information, and internship dates), renders an HTML template with those values, and converts it to a PDF using WeasyPrint. The resulting file is stored at uploads/agreements/ and linked to the InternshipAgreement model. Students and admins can then download it through dedicated endpoints.

```python
def generate_internship_agreement(internship, admin=None):
    """Render the internship agreement template, generate a PDF, and save it."""
    if not isinstance(internship, Internship):
        raise ValueError('A valid Internship instance is required.')

    internship = Internship.objects.select_related(
        'student',
        'student__university',
        'company',
    ).get(pk=internship.pk)

    if internship.student.university is None:
        raise ValueError('The internship student must belong to a university.')

    if admin is None:
        admin = Admin.objects.filter(
            university=internship.student.university,
        ).order_by('id').first()

    if admin is None:
        raise ValueError('No administrator found for this university.')

    generated_date = timezone.now()
    context = {
        'internship': internship,
        'admin': admin,
        'reference_number': f'CS-{internship.id}/{generated_date.year}',
        'academic_year': _build_academic_year(internship.start_date),
        'duration': _format_duration(internship.start_date, internship.end_date),
        'generated_date': generated_date,
        'university_logo_base64': _get_university_logo_base64(
            internship.student.university,
        ),
    }
```

```python
def generate_internship_agreement(internship, admin=None):
    html_string = render_to_string(
        'administration/convention_template.html',
        context,
    )
    pdf_bytes = HTML(string=html_string, base_url=str(settings.MEDIA_ROOT)).write_pdf()

    agreement, _ = InternshipAgreement.objects.get_or_create(internship=internship)
    if agreement.pdf_file:
        agreement.pdf_file.delete(save=False)

    agreement.pdf_file.save(
        f'internship_agreement_{internship.id}.pdf',
        ContentFile(pdf_bytes),
        save=False,
    )
    agreement.save()

    return agreement
```

Figure 35: PDF agreement generation service (administration/services.py)

4.5 Database Structure

The Stag.io database is managed by PostgreSQL 13. Django's ORM handles all migrations and schema creation automatically from the model definitions in core/models.py and student/models.py. The following tables form the core structure of the application:

Table Name	Django App	Description
core_user	core	Base user table — email login, role, active status
core_student	core	Extends core_user — full_name, wilaya, github_link, portfolio_link, profile_image
core_company	core	Extends core_user — name, description, wilaya, website, logo
core_admin	core	Extends core_user — department, title, university FK
core_university	core	University institution — name, code, wilaya, address, logo
core_internshipoffer	core	Internship posting — title, description, location, wilaya, type, salary, dates, status, photo
core_internshipApplication	core	Student application— links student, company, offer, tracks status through workflow
core_internshipagreement	core	Generated PDF document — linked one-to-one with a validated Internship
core_notification	core	In-app notifications for all user roles — message, link, is_read, type
core_offerskill	core	Junction table — links InternshipOffer to required Skills (many-to-many)
student_skill	student	Master list of all available technical skills (pre-seeded, 30 entries)
student_studentskill	student	Junction table — links Student to selected Skills (their profile chips)

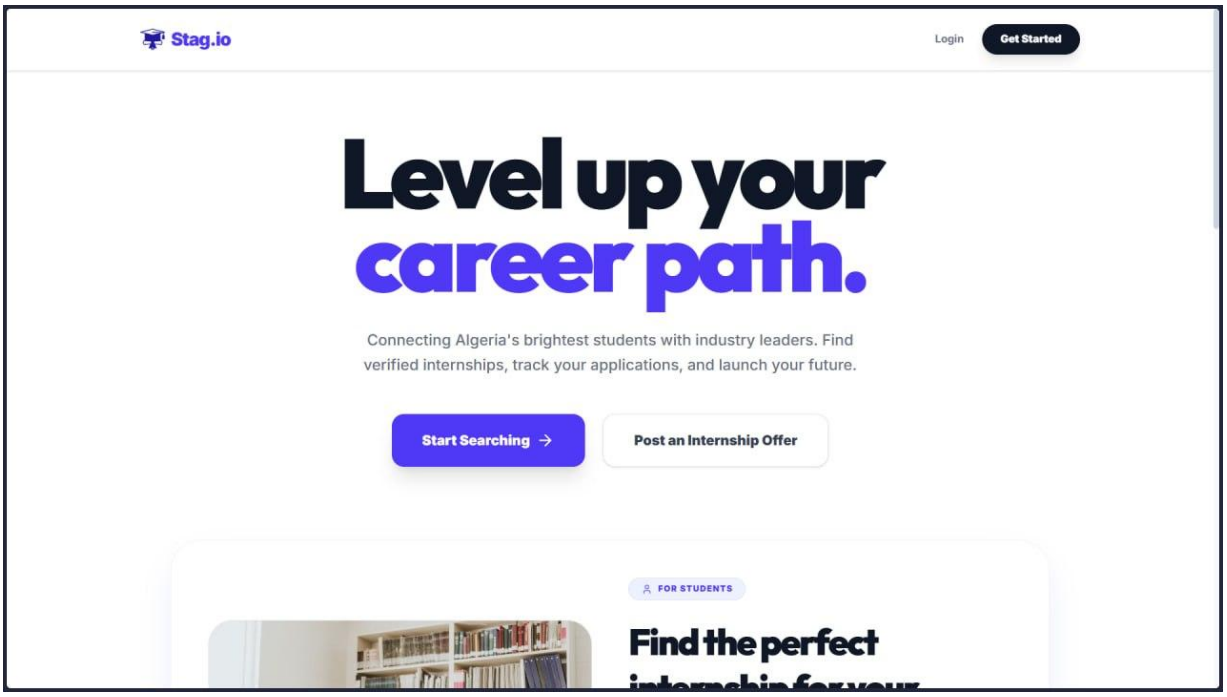
Figure 36: Database Tables Overview

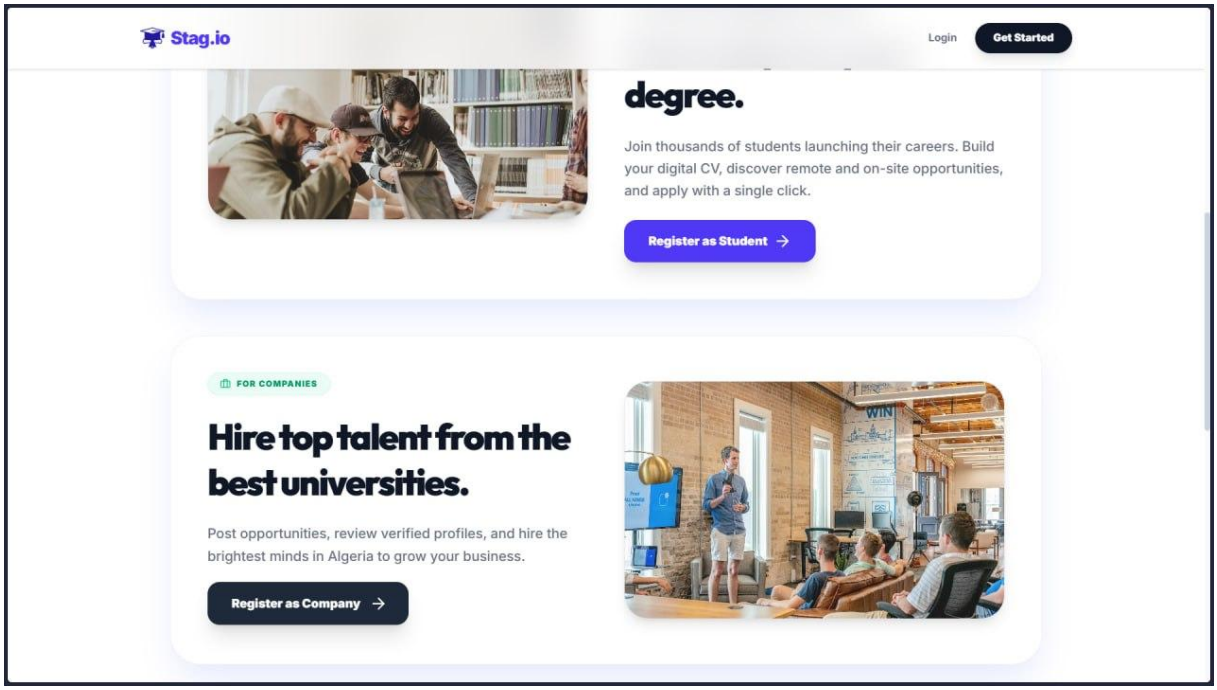
4.6 Application Interfaces

This section showcases the important interfaces of Stag.io

4.6.1 Landing Home Page:

Serves as a welcoming page to unregistered users to Stag.io, simple and informative with the goal of guiding new users through the application.





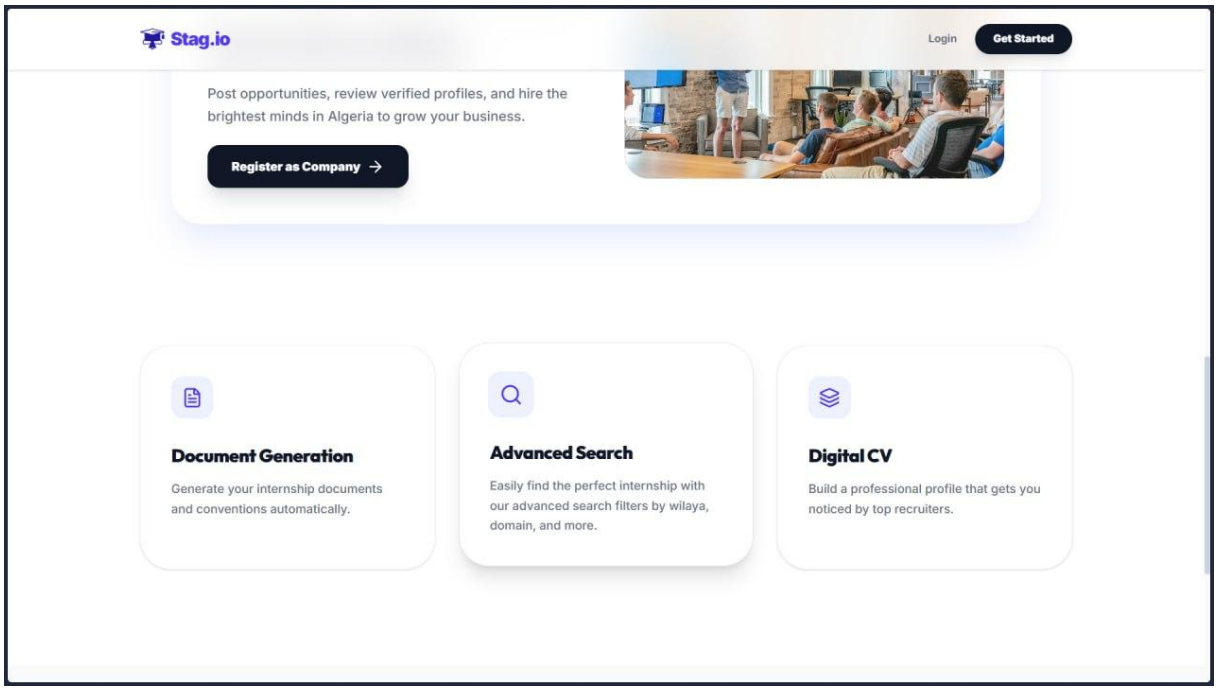


Figure 36: Landing Home Page

4.6.2 Authentication – Login and Register Page:

The authentication interface serves as the entry point of the platform. Users can register as a student or as a company and log in with their credentials. Student registration requires a university email address.

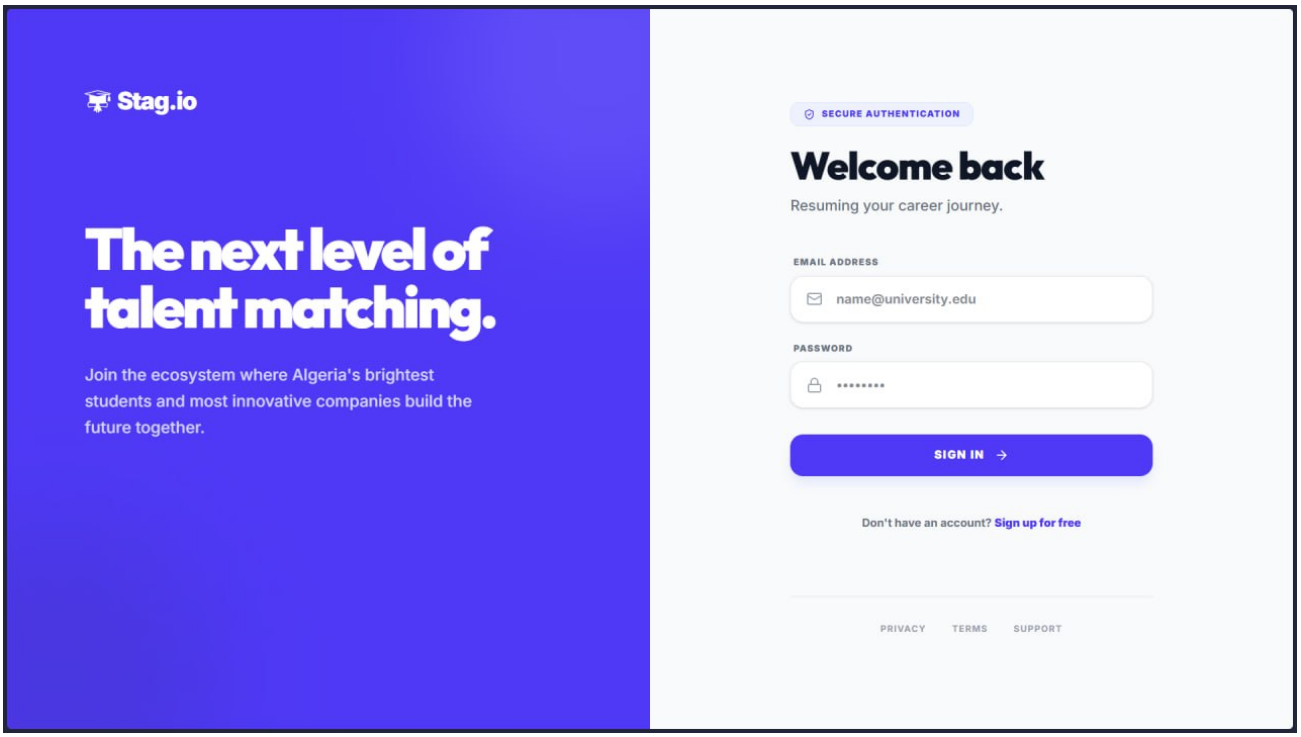


Figure 37: Login Page

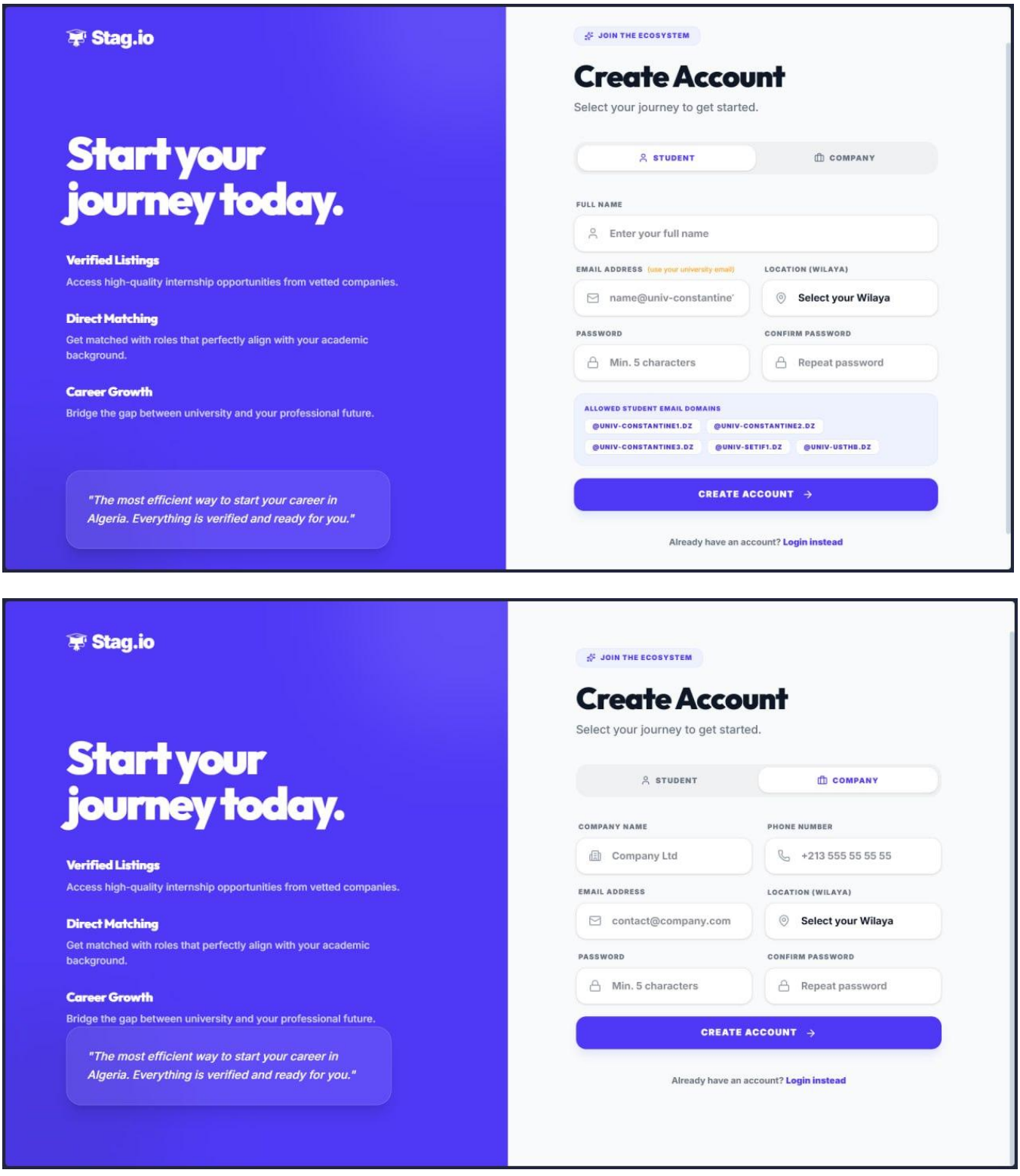


Figure 38: Register Page

4.6.3 Student Dashboard:

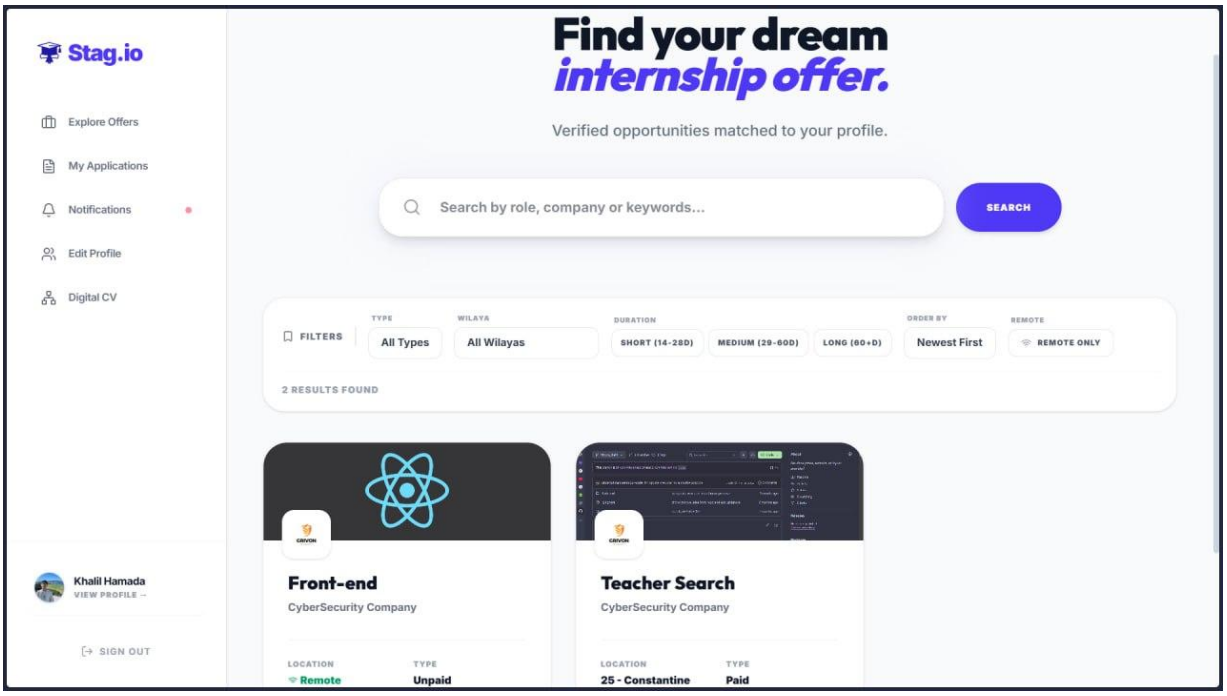


Figure 39: Student Dashboard

4.6.4 Student My Application Page:

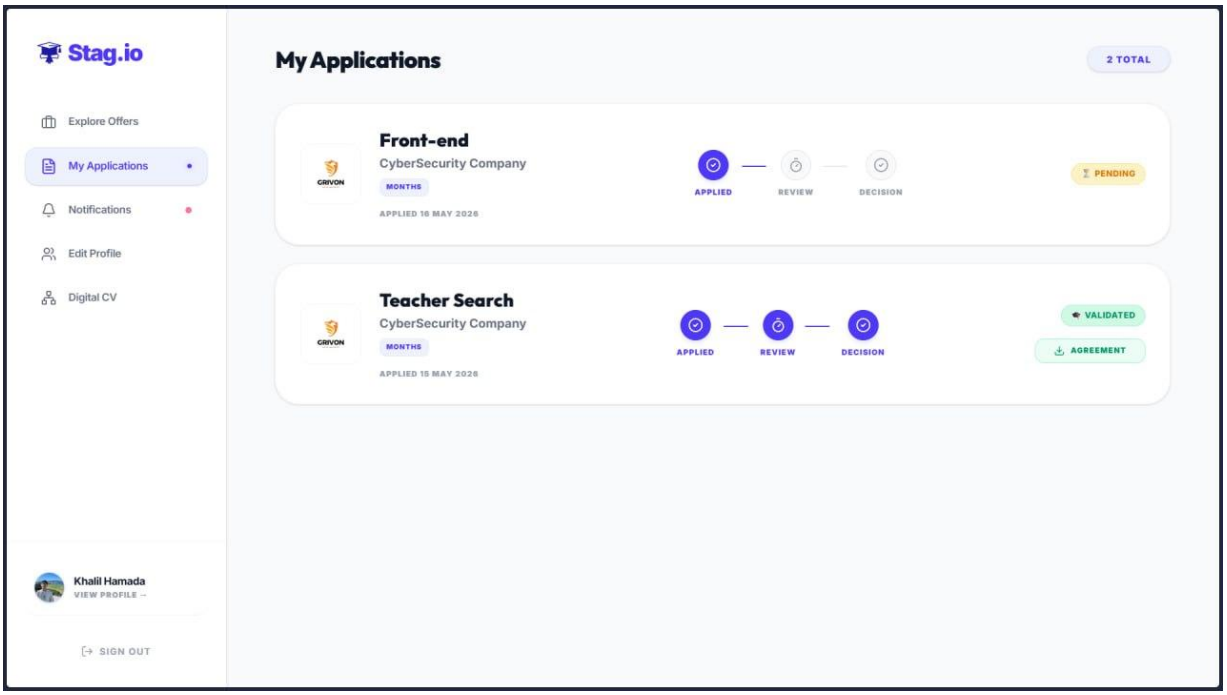


Figure 40: Student My Application Page

4.6.5 Student Edit Profile Page:

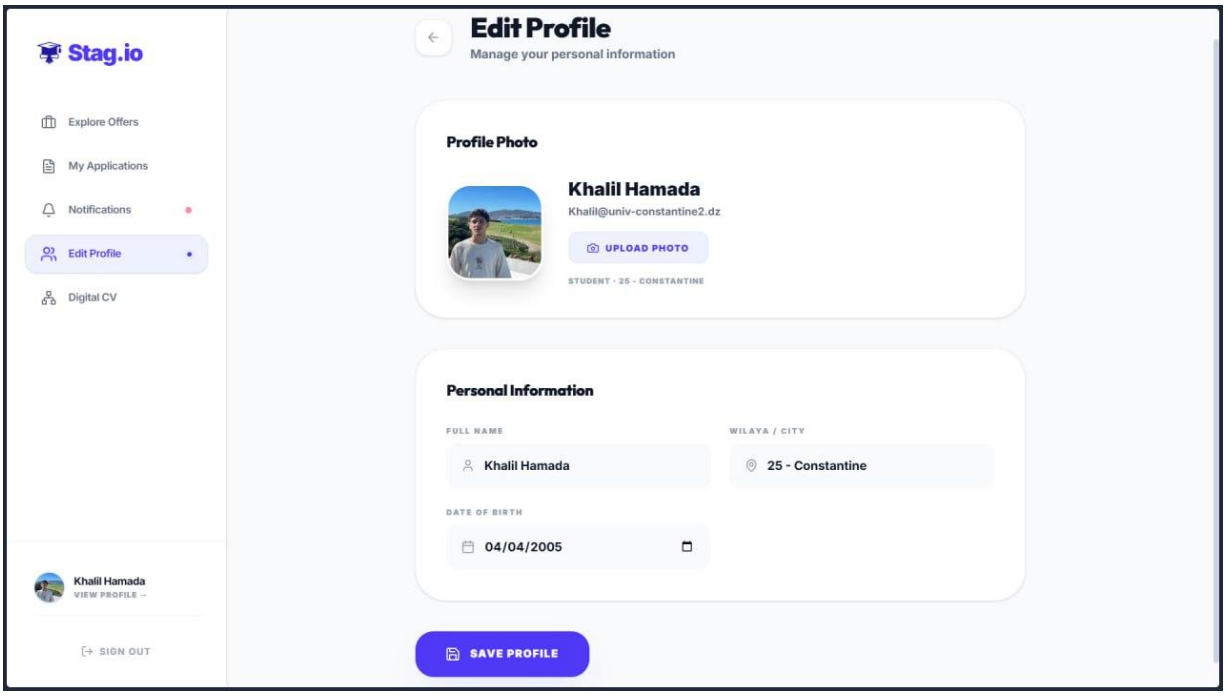


Figure 40: Student Edit Profile Page

4.6.6 Student Digital CV Page:

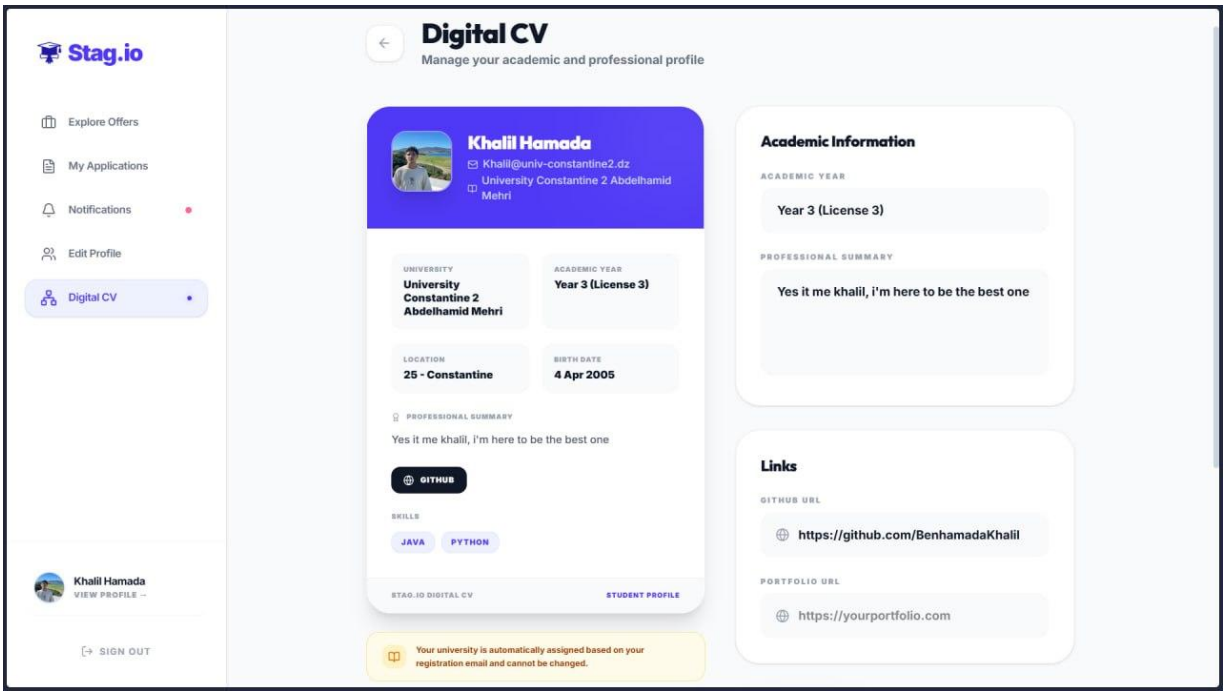


Figure 41: Student Digital CV Page

4.6.7 Student Internship Offer Interface:

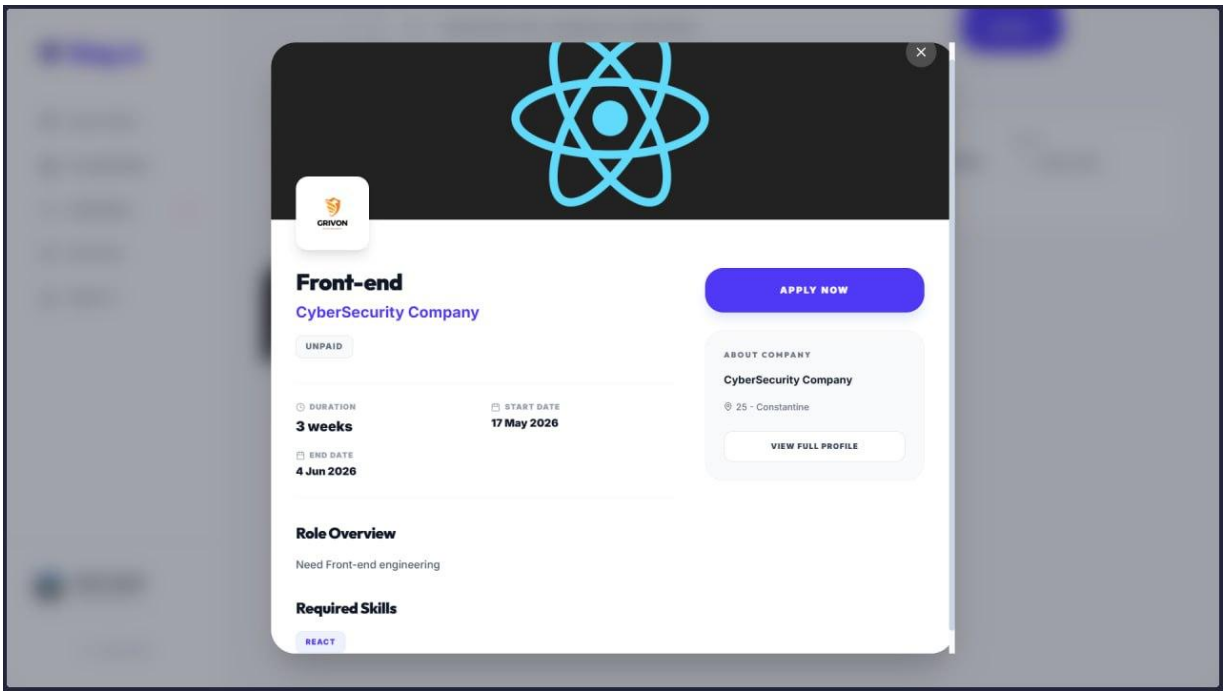


Figure 42: Student Internship Offer Interface

4.6.8 Company Dashboard:

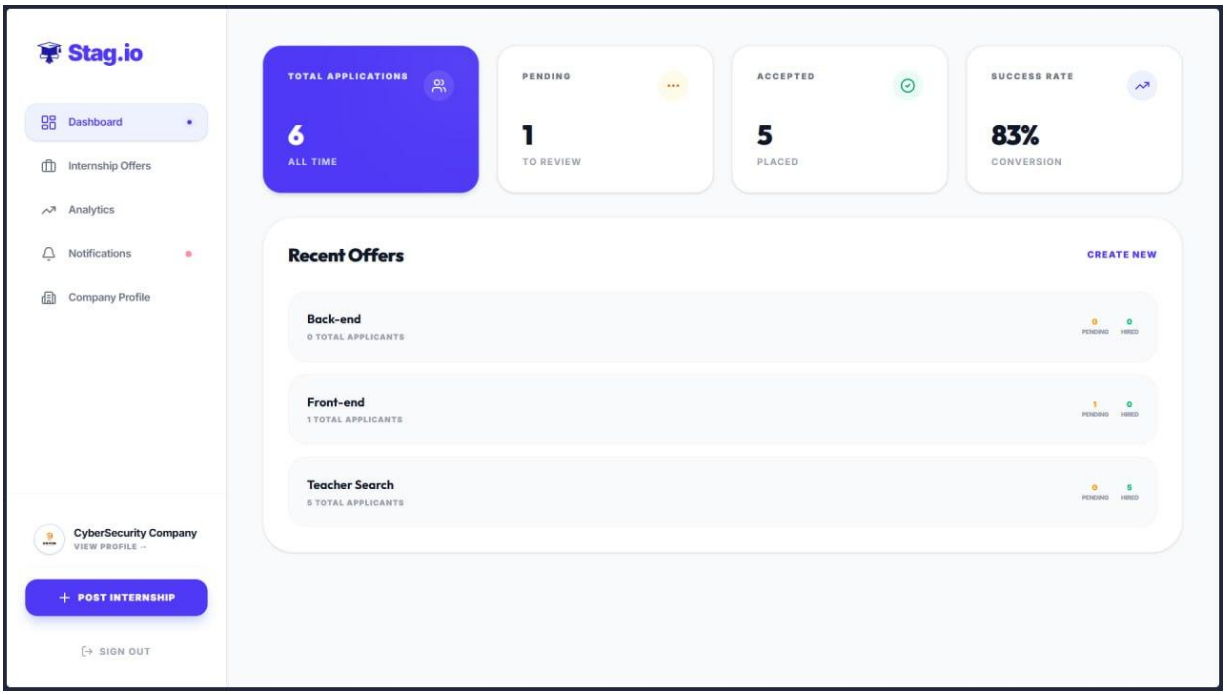


Figure 43: Company Dashboard

4.6.9 Company Internships Offers Page:

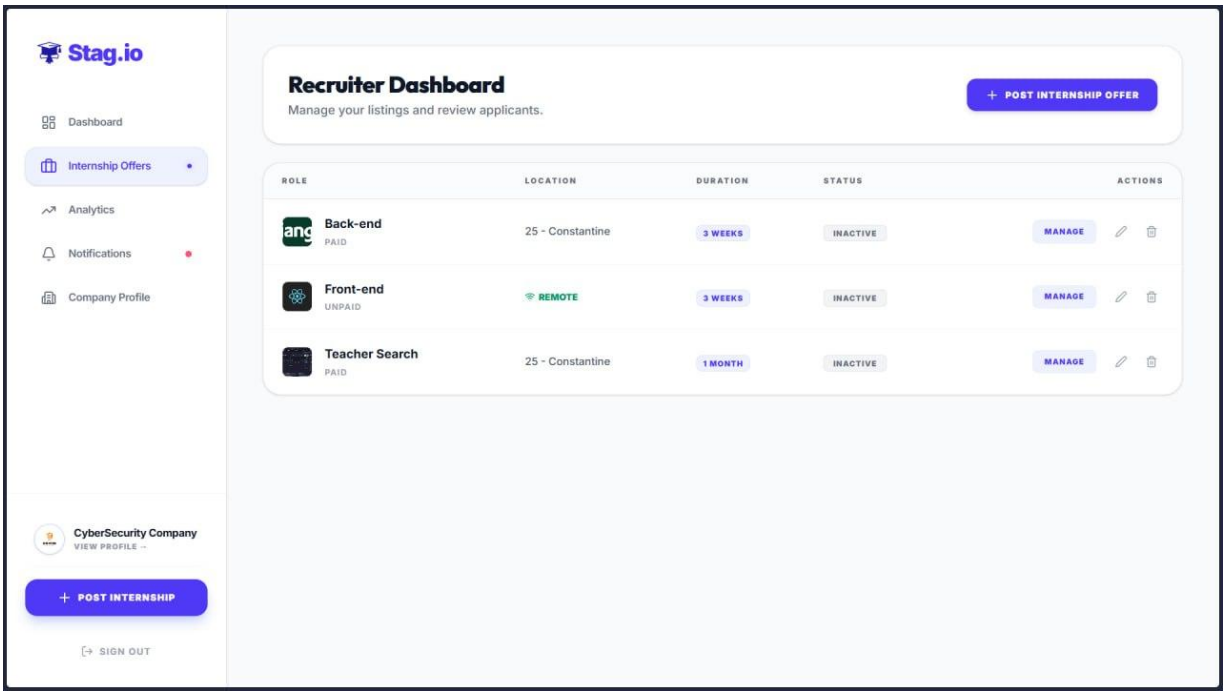


Figure 44: Company Internships Offers Page

4.6.10 Company Applicant Management Interface:

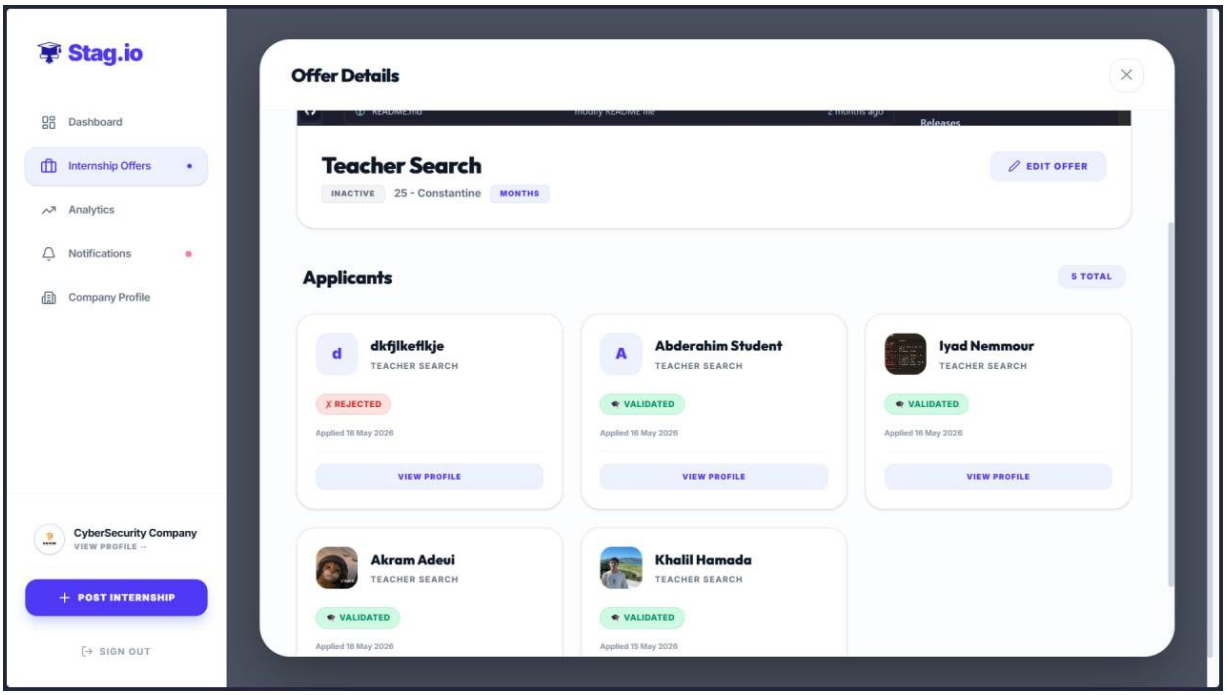
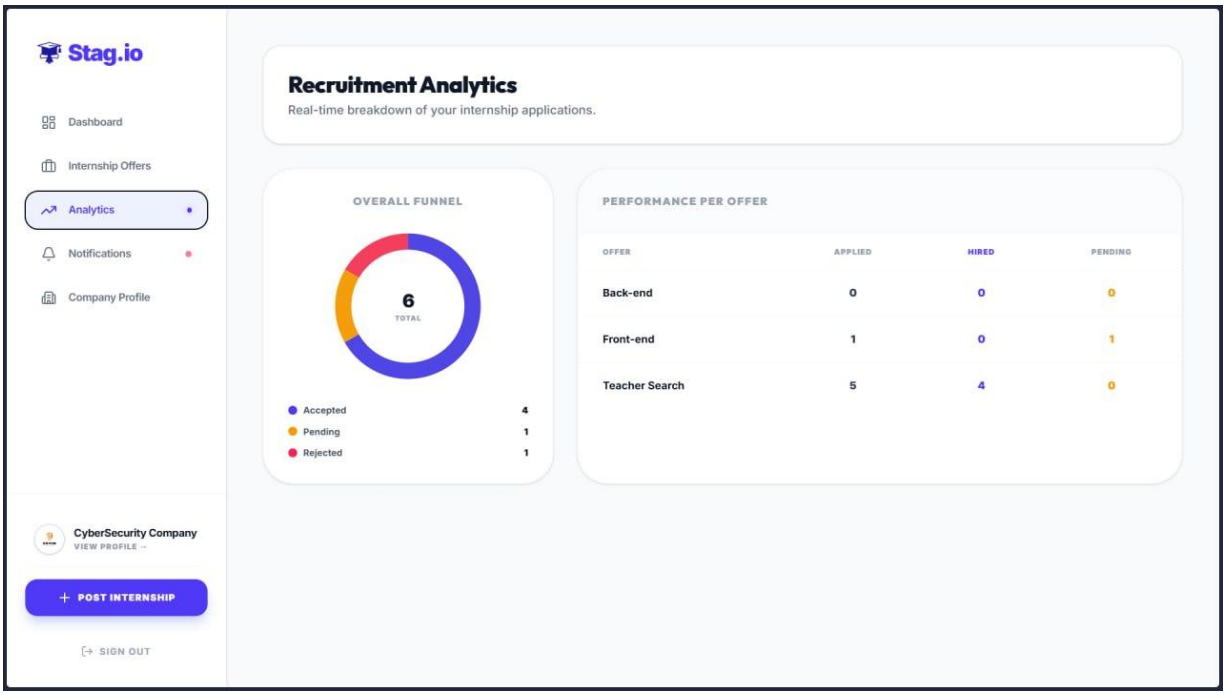


Figure 45: Company Applicant Management Interface

4.6.11 Company Analytics Page:



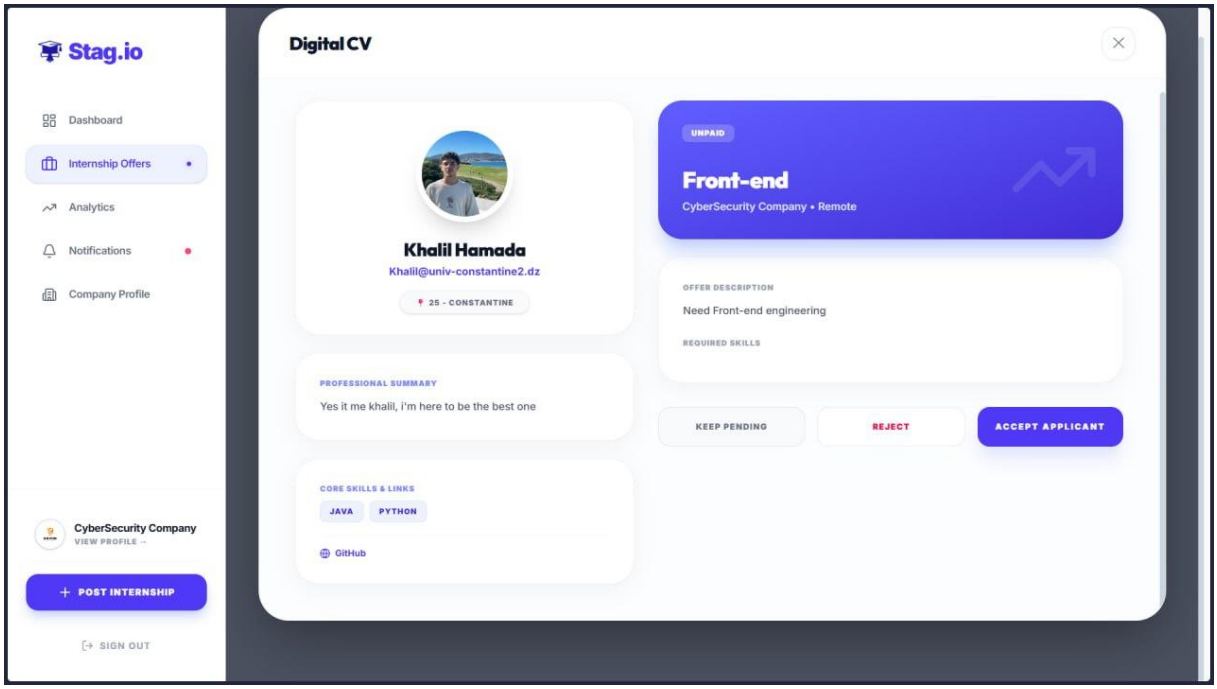


Figure 46: Company Analytics Page

4.6.12 Admin Dashboard:

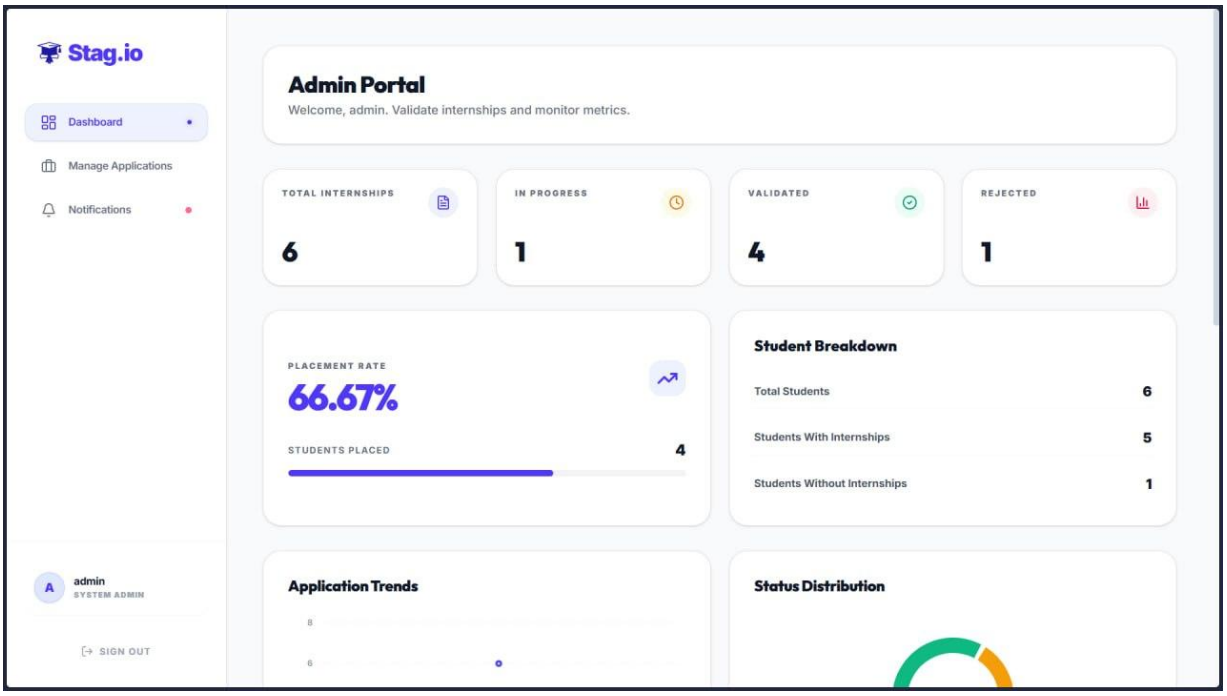


Figure 47: Admin Dashboard

4.6.13 Admin Manage Applications Page:

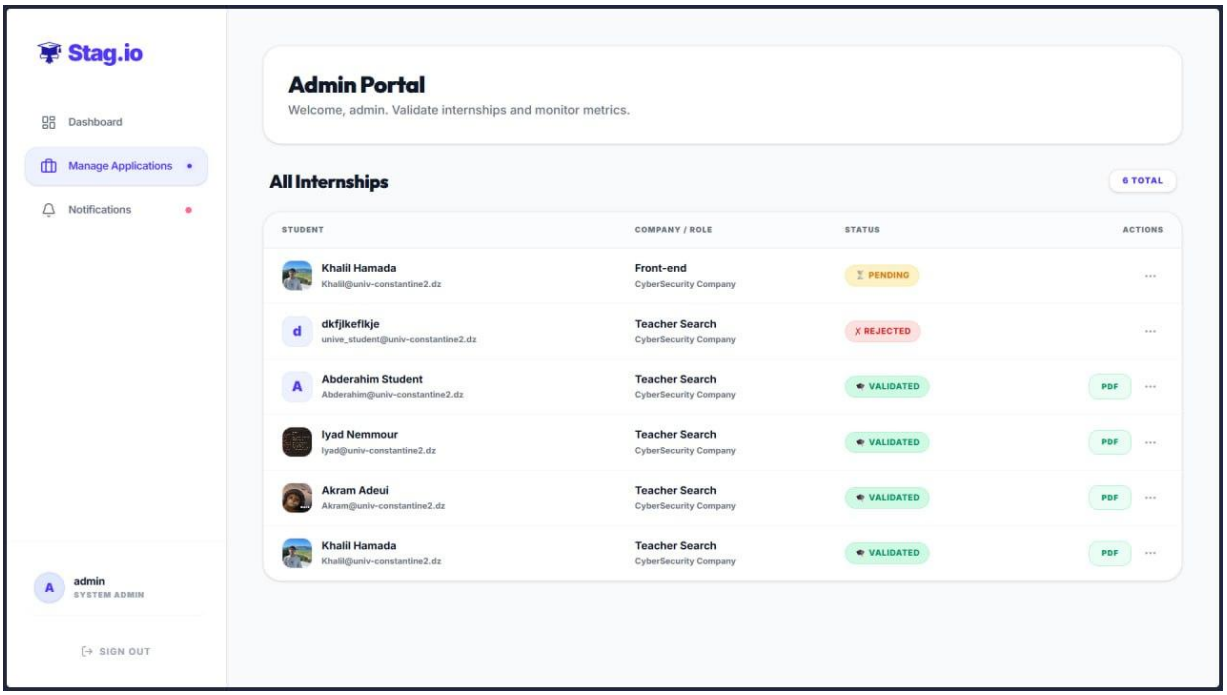


Figure 48: Admin Manage Applications Page

4.6.14 Notifications Page:

Each user has their own Notifcation Page, and each one has the same theme and premise.

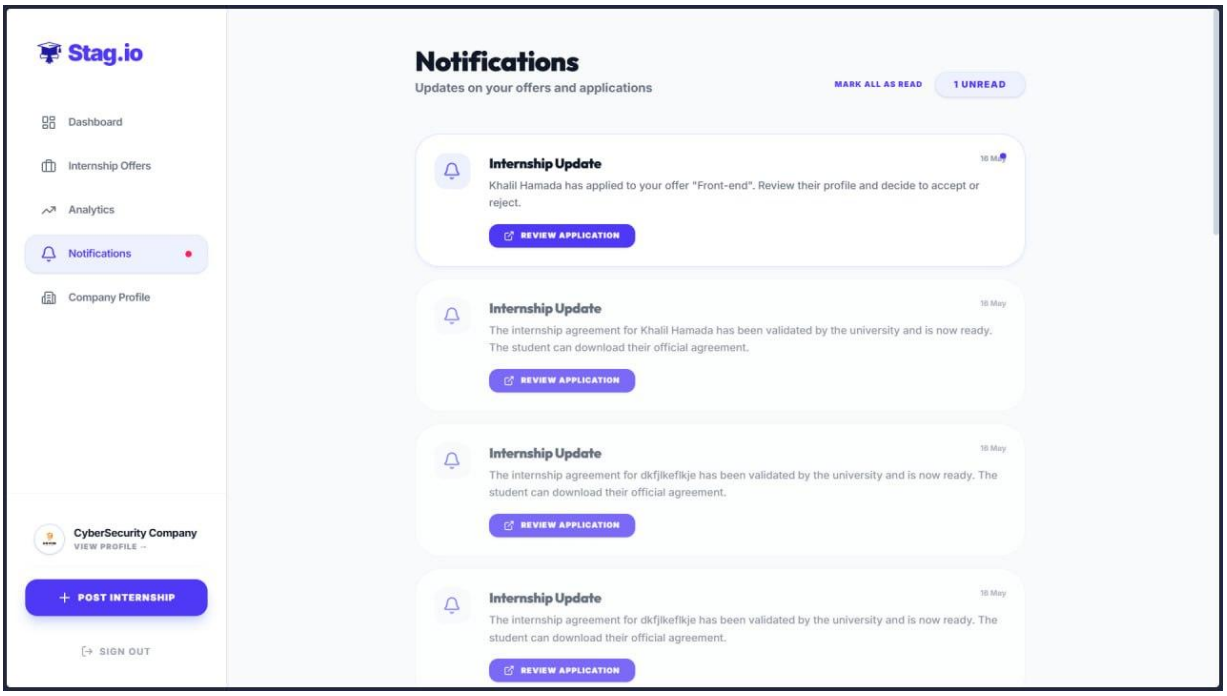


Figure 49: Notification Page

4.7 Conclusion

This chapter presented the implementation of the Stag.io platform in full detail. We described the three-tier architecture that separates the React frontend, the Django REST API backend, and the PostgreSQL database. We listed all hardware and software tools used, including their versions and roles in the project.

We presented representative code examples covering the user model hierarchy, the internship offer with computed duration, role-based permission enforcement, the multi-filter search system, the centralized notification service, and the PDF agreement generation pipeline.

We then described the complete database schema, explaining the multi-table inheritance design for users and the junction tables used for the many-to-many relationships between skills and both students and offers.

Finally, we walked through the key user interfaces for all three actors (Student, Company, and Administration) demonstrating how the functional requirements defined in Chapter 2 and the design diagrams from Chapter 3 have been fully realized in the working application.

General Conclusion

Summary of Achievements:

This project set out to address a real and recurring problem faced by students, companies, and university administrators in Algeria: the inefficiency, opacity, and fragmentation of the traditional internship management process. Through the design and implementation of Stag.io, a full-stack web application, we have successfully delivered a centralized platform that covers the entire lifecycle of a university internship from offer discovery and application, through company review and acceptance, to administrative validation and official document generation.

The platform was built on a modern three-tier architecture: a React.js 18 frontend, a Django REST Framework backend, and a PostgreSQL 13 relational database. Role-based access control enforced via JSON Web Tokens ensures that students, companies, and administrators each operate within their dedicated space, with no unauthorized cross-access. The multi-criteria search engine, centralized notification service, and automated PDF generation of the Convention de Stage together represent the core functional achievements of the project.

All three actor interfaces, the Student Dashboard, the Company Recruiter Portal, and the Admin Portal were realized and tested against the functional requirements defined in Chapter 2. The use-case scenarios, activity diagrams, sequence diagrams, and relational model elaborated in Chapter 3 were faithfully translated into working code, as demonstrated in the implementation chapter.

Challenges Encountered:

Several technical and organizational challenges arose during the development of Stag.io. On the technical side, configuring the WeasyPrint PDF generation library on Windows required installing the GTK3 runtime separately, which added complexity to the development environment setup. Designing a database schema that correctly implements multi-table inheritance for the three user roles while keeping migrations clean and conflict-free was another non-trivial task.

The multi-criteria filtering endpoint for internship offers, combining text search, wilaya filters, skill chips, duration ranges, and ordering required careful use of Django ORM annotations and ExpressionWrapper constructs to ensure acceptable response times. Synchronizing frontend state across the three dashboards, particularly for real-time notification badges, also required thoughtful component architecture in React.

On the organizational side, coordinating the work of three team members across backend, frontend, and documentation required structured version control practices and clear interface contracts between the API and the UI. Regular integration testing using Swagger UI helped catch mismatches early.

Objectives Achieved:

➢ **Centralized internship platform:** A single web application replaces fragmented email chains, paper forms, and disconnected tools for all three actor types.

➢ **Administrative automation:** The Convention de Stage PDF is generated automatically upon admin validation, pre-filled with all required data, eliminating manual paperwork.

➢ **Transparent tracking:** Students can follow the exact status of every application through a visual three-step pipeline (Applied → Review → Decision), with in-app notifications at each transition.

➢ **Analytics and reporting:** The Admin and Company dashboards provide real-time metrics on placements, application funnels, and student breakdown statistics.

Future Perspectives:

While Stag.io achieved the goals defined at the outset of the project, several enhancements remain as natural extensions for future work:

➢ **AI-powered matching:** Integrating a recommendation engine that scores and ranks offers for each student based on profile similarity and historical application outcomes, beyond simple skill-chip overlap.

➢ **Mobile application:** Developing native iOS and Android clients using the existing Django REST API as a backend, extending platform accessibility to mobile users.

➢ **Digital signature integration:** Incorporating an e-signature workflow into the Convention de Stage generation pipeline, replacing the current download-and-sign process with a fullypaperless flow

➢ **Real-time messaging:** Adding a WebSocket-based chat module so students and company recruiters can communicate directly within the platform.

In conclusion, Stag.io demonstrates that a well-architected, role-driven web application can meaningfully reduce administrative friction, improve transparency, and bring students and companies closer together in the internship ecosystem. The platform lays a solid technical foundation on which future features can be built incrementally, making it a long-term asset for university internship management in the Algerian higher education context.

Bibliographic References

[1] Django Software Foundation. Django Documentation — Version 3.2. Available at: https://docs.djangoproject.com/en/3.2/ [Accessed May 2026].

[2] Django REST Framework. Django REST Framework Documentation. T. Christie. Available at: https://www.django-rest-framework.org/ [Accessed May 2026].

[3] Simple JWT. Simple JWT — A JSON Web Token authentication plugin for DRF. Available at: https://django-rest-framework-simplejwt.readthedocs.io/ [Accessed May 2026].

[4] drf-spectacular. OpenAPI 3 Schema Generation for Django REST Framework. Available at: https://drf-spectacular.readthedocs.io/ [Accessed May 2026].

[5] React. React — A JavaScript library for building user interfaces. Meta Platforms, Inc. Available at: https://react.dev/ [Accessed May 2026].

[6] Vite. Vite Next Generation Frontend Tooling. Available at: https://vitejs.dev/ [Accessed May 2026].

[7] Axios. Axios HTTP Client Documentation. Available at: https://axios-http.com/ [Accessed May 2026].

[8] Tailwind CSS. Tailwind CSS — A utility-first CSS framework. Tailwind Labs. Available at: https://tailwindcss.com/ [Accessed May 2026].

[9] PostgreSQL Global Development Group. PostgreSQL 13 Documentation. Available at: https://www.postgresql.org/docs/13/ [Accessed May 2026].

[10] WeasyPrint. WeasyPrint — The Awesome Document Factory. Available at: https://doc.courtbouillon.org/weasyprint/ [Accessed May 2026].

[11] LinkedIn Corporation. LinkedIn — Professional Networking Platform. Available at: https://www.linkedin.com/ [Accessed April 2026].

[12] Indeed. Indeed — Job Search Engine. Available at: https://www.indeed.com/ [Accessed April 2026].

[13] Emploitic. Emploitic — Algerian Job & Internship Platform. Available at: https://www.emploitic.com/ [Accessed April 2026].

[14] Ministry of Higher Education and Scientific Research, Algeria. Guidelines for University Internship Procedures. Algiers, 2022.

[15] pgAdmin Development Team. pgAdmin 4 Documentation. Available at: https://www.pgadmin.org/docs/ [Accessed May 2026].

[16] Visual Studio Code. VS Code Documentation. Microsoft Corporation. Available at: https://code.visualstudio.com/docs/ [Accessed May 2026].

[17] Astah. Astah UML — UML Modeling Tool. Change Vision Inc. Available at: https://astah.net/ [Accessed May 2026].